UNIT- I PYTHON PROGRAMMING FUNDAMENTALS

Introduction – Features of Python – References and Keywords - Literals - String – Numeric, Boolean and Special Literals - List, Tuple, Set and Dictionary Data types – Operators – Membership and Identity Operators - Conditional and Control Constructs.

1.1 Introduction to Python Programming

What is a program?

A program is a sequence of Executable instructions that perform specific task .
There are few basic instructions appear in every programming language:

input: Get data fromthe keyboard, a file, or some other device.
Process: Perform Arithemtic & logical operation
output: Display data on the screen or send data to a file or other device.

Python was **created in 1980s by Guido van Rossum**. During his research at the **National Research Institute for Mathematics and Computer Science in the Netherlands**. The inspiration for the name came from the BBC's TV Show – ' **Monty Python's Flying Circus'** , as he was a big fan of the TV show and also he wanted a short, unique and slightly mysterious name for his invention and hence he named it Python

What is python: Python is a set of instructions that we give in the form of a Program to our computer to perform any specific task. It is high-level Programming language interpreted and object-oriented.

1.2 Features of Python

Simplicity and Readability: Python's syntax is clean and straightforward, resembling everyday English. This approach minimizes the learning curve and allows programmers to focus on solving problems rather than getting bogged down by complex syntax rules.

Interpreted Language: As an interpreted language, Python executes code line by line, which makes debugging easier and speeds up development time. You don't need to compile your programs before running them.

Dynamically Typed: Variables in Python don't require explicit declaration to reserve memory space. The declaration happens automatically when you assign a value to a variable. This dynamic typing adds flexibility to your code.

Object-Oriented and Procedural Paradigms: Python supports both object-oriented and procedural programming styles. This means you can use classes and objects or write procedures and functions, giving you the flexibility to choose the best approach for your project.

Extensive Standard Library: Python comes with a rich standard library that provides modules and functions for various tasks—handling web protocols, string operations, system functionalities, and more. This vast library reduces the need to write code from scratch.

Cross-Platform Compatibility: Write your code once and run it anywhere. Python is compatible with major platforms like Windows, macOS, and Linux, allowing your programs to be platform-independent.

Open Source and Community Support: Being open-source, Python's source code is freely available to the public, encouraging a collaborative environment. A vibrant community contributes to a wealth of resources, libraries, and frameworks.

Integration Capabilities: Python can easily integrate with languages like C, C++, and Java. This interoperability makes it a preferred choice for projects requiring performance optimization.

High-Level Language:
Python abstracts away many of the complex details of the computer, such as memory management. This means you can focus on programming logic rather than low-level processes.

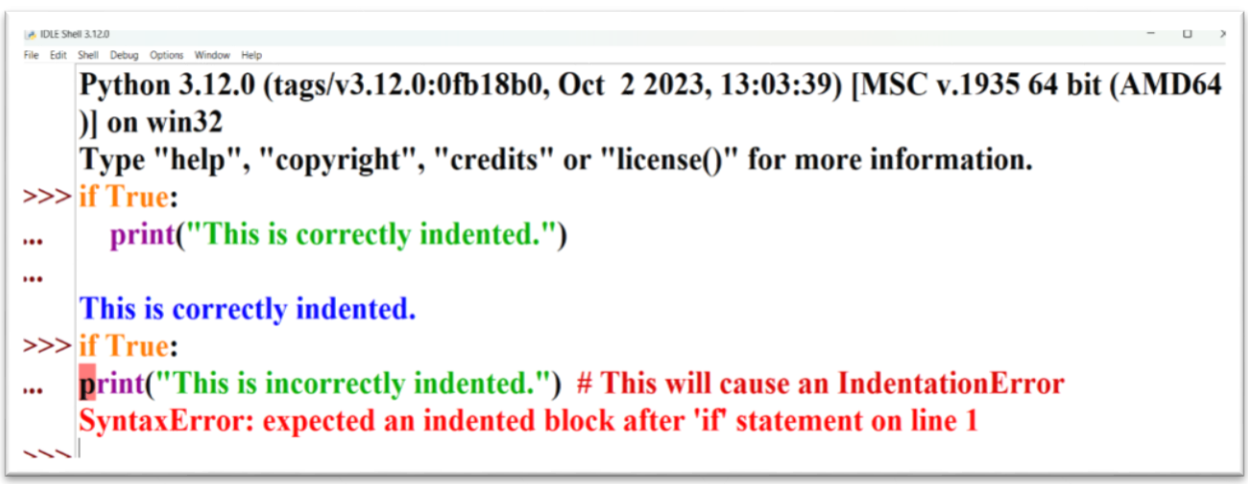
Versatile Applications: Python is used in web development, data science, machine learning, artificial intelligence, automation, scripting, and more. Its versatility stems from its simplicity and the power of its libraries.

1.3 Running Python Code: Running Python code involves two main phases: compilation and execution.

1.3.1 Compilation: Python is an interpreted language and doesn't have a traditional compilation phase like C or Java. Instead, it compiles the source code into bytecode (.pyc files) before execution. During this process, proper indentation is crucial as it helps the interpreter understand the structure of the code.

Indentation is Most of the programming languages like C, C++, Java use braces { } to define a block of code, but Python uses indentation. Proper indentation is crucial as it defines the structure and flow of the code. Incorrect indentation will lead to an Indentation Error

#Example for indentation



1.3.2. Execution: Once the code is compiled to bytecode, it is executed by the Python interpreter. There are two primary modes of execution in Python: Interactive Mode and Script Mode. You can use IDLE for both interactive mode and script mode. Let's see how to use these modes in IDLE Shell 3.12. IDLE stands for **Integrated Development and Learning Environment**. It provides an easy-to-use interface for writing, running, and debugging Python code. It includes features like syntax highlighting, auto-completion, and a built-in Python shell for interactive mode

Interactive Mode allows you to type and run Python commands directly in the Python interpreter.

(a).Open IDLE Shell: Launch IDLE by searching for "IDLE" in your start menu or applications folder. The IDLE Shell window will open, displaying the >>> prompt.

(b).Type and Run Commands: You can type Python commands directly at the >>> prompt. Press Enter to execute the command.

#Example:

```
>>> print("Hello, world!") Hello, world!
```

In this mode, you can immediately see the results of your commands. It's great for quick tests and learning.

Script Mode involves writing your Python code in a file and then running that file.

(a) Write the Script: Open IDLE and click on File > New File to open a new editor window. Write your Python code in the editor window.

#Example

```
# script.py print("Hello, world!")
```

(b) Save the Script: Save the file with a .py extension by clicking on File > Save or File > Save As... and choosing a location and file name (e.g., script.py).

(c) Run the Script: In the editor window, click on Run > Run Module or simply press F5. This will execute the script in the IDLE Shell window.

Running the script in IDLE will display the output:

```
Hello, world!
```

Note: IDLE Shell 3.12 makes it easy to switch between interactive and script modes for different tasks.
Interactive Mode: Type and run commands directly in the IDLE Shell (>>> prompt).
Script Mode: Write and save your code in a .py file, then run it using Run > Run Module or F5 in IDLE.

1.4 Python Tokens: Python Tokens are the smallest units of a Python program that have meaningful value to the interpreter. They are the building blocks of the language, used to construct statements and expressions.

1. **Keywords**: Reserved words that have a special meaning in Python. They cannot be used as identifiers. Examples include if, else, while, for, def, class, import, etc.
2. **Identifiers**: Names used to identify variables, functions, classes, modules, etc. They must start with a letter (A-Z or a-z) or an underscore (_) followed by letters, digits (0-9), or underscores. Example: myVariable, calculatesum.
3. **Literals**: Constant values used in the program. They include:
   - **String Literals**: "Hello, World!", 'Python'
   - **Numeric Literals**: 42, 3.14
   - **Boolean Literals**: True, False
   - **Special Literals**: None
4. **Operators**: Symbols that perform operations on variables and values. Examples:
   - **Arithmetic Operators**: +, -, *, /, %, **

- **Comparison Operators**: ==, !=, >, <, >=, <=
- **Logical Operators**: and, or, not
- **Assignment Operators**: =, +=, -=, *=, /=
- **Bitwise Operators**: &, |, ^, ~, <<, >>

5. **Delimiters**: Symbols that separate elements and structures. Examples:
   - **Parentheses**: ()
   - **Brackets**: []
   - **Braces**: {}
   - **Comma**: ,
   - **Colon**: :
   - **Semicolon**: ;
   - **Dot**: .
6. **Escape Sequences**: Represent special characters within string literals. Examples:
   - \n: Newline
   - \t: Horizontal tab
   - \\: Backslash
   - \': Single quote
   - \": Double quote

1.5 Python Statements and Expressions

15.1 Statement: There are two types of statements in Python: executable statements and non-executable statements.

1.5.1.1 Executable Statements in Python: Executable statements are sequences of instructions that the Python interpreter can execute. They perform an action or a sequence of actions. Statements are the building blocks of a Python program. There are two basic types of executable statements based on when and where input is given during their processing:

Assignment Statements : These are processed during the compilation phase. However, Python does not compile code in the traditional sense; instead, it interprets the code and converts it into bytecode (.pyc files). In Python, you write the code, save it, and run it with the input given in the source code. Example code

```
x = 5  # Assignment statement print(x)

#output: 5
```

Input Statements: These are processed during the execution phase, where the Python interpreter processes the statements line by line. In this case, you save and run the code, and the input is provided during execution. Example code

```
name = input("Enter your name: ")  # Input statement print(f"Hello, {name}!")

#output
Enter your name: John Hello,
John!
```

15.1.2Non-executable statements are comment statements in a Python program. They are ignored by the interpreter and are used to add notes, explanations, or descriptions within the code to make it more readable and understandable..

There are two types of comment statements in Python:

Single-Line Comments: Start with a hash symbol (#). Everything after the # on that line is considered a comment. Example:

```
# This is a single-line comment
```

Multi-Line Comments: Enclosed in triple quotes, either ''' or """. These comments can span multiple lines.

```
'''
This is a multi-line comment.
You can write as many lines as you need.
It is enclosed in triple quotes.
'''
```

15.2 Expression:: An expression is a combination of values, variables, operators and function calls that produces a value. Expressions are evaluated by the interpreter and return a value.**Example**: Arithmetic expressions.

```
y = 2 + 3   # Arithmetic expression
```

Note: Statement vs. Expression:

- A statement performs an action and **does not return a value**. For example, `x = 5`is a statement because it assigns the value `5`to the variable `x`without returning a value.
- An expression is a combination of elements that **evaluates to a value**. For example, `2 + 3`is an expression because it computes the sum of `2`and `3`, returning the value `5`.

1.6 References in Python

In Python, A variable is an identifier used to store a value in a specific memory location , variables are like labels attached to values stored in the computer's memory. When you assign a value to a variable, you're creating a reference to that value.

Simple Example:

```
# Assign the same value to multiple variables x = 5
y = 5
z = 5
# Print the  same memory addresses for  x,y, z variables print("Memory address of x:",
id(x))
print("Memory address of y:", id(y)) print("Memory address
of z:", id(z))

# Change the values of the variables x = 10
y = 15

# Print the different memory addresses after changing values print("Memory address of x
after changing to 10:", id(x)) print("Memory address of y after changing to 15:", id(y))
print("Memory address of z (unchanged):", id(z))
```

Output
Memory address of x: 9793216 Memory address of y: 9793216 Memory address of z: 9793216
Memory address of x after changing to 10: 9793376 Memory address of y after changing to 15: 9793536 Memory address of z (unchanged): 9793216

Explanation:

- **Memory Optimization**:

Python reuses memory addresses for immutable objects with the same value.
Variables x, y, and z initially reference the same memory location for the value 5.

- **Value Changes and New References**:

Changing the value of x to 10 and y to 15 creates new memory objects.
x and y now reference new memory addresses, separate from the original address of 5.

- **Memory Stability for Unchanged Variables**:

The variable z retains its original memory address since its value remains 5.
Unchanged variables continue to reference the same memory location.

1.7 Keywords in Python

In programming languages, **keywords** are **reserved words** that have special meanings and purposes. They are part of the language's syntax and structure. Keywords are used to perform specific operations and to control the flow of a program. As of **Python 3.12**, there are **37 keywords**.

Here is the complete list of Python keywords:

False, None, True, and, as, assert, async, await, break, case, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, match, nonlocal, not, or, pass, raise, return, try, while, with, yield

Important Points:

- **Cannot be used as identifiers:** Keywords **cannot** be used as names for variables, functions, classes, or any other user-defined items.
- **Fixed meanings:** The meaning of a keyword is predefined in the language and cannot be changed by the programmer.
- **Essential for syntax:** Keywords help define the structure and organization of a program.

1.8 Literals in Python:

In Python, **literals** are fixed values that you write directly in your code. They represent basic data types and are used to assign values to variables. Python has several types of literals: **Numeric Literals, String Literals, Boolean Literals, and the Special Literal (None).**

1.8.1 Numeric Literals: Numeric literals represent numbers. They can be:

Integer Literals: Whole numbers without a decimal point.
Float Literals: Numbers with a decimal point.
Complex Literals: Numbers with a real and imaginary part.

Examples:

```
# Integer literal age = 25
print(age)
# Output: 25

# Float literal price =
19.99 print(price)
# Output: 19.99

# Complex literal
complex_number = 3 + 4j
print(complex_number)  #
Output: (3+4j)
```

1.8.2 String Literals:

String literals represent text enclosed in single or double quotes.

Examples:

```
# String literal with single quotes greeting = 'Hello,
World!' print(greeting)

# Output: Hello, World!

# String literal with double quotes name = "Alice"
print(name)
# Output: Alice
```

1.8.3. Boolean Literals:

Boolean literals represent truth values: True or False.

Examples:

```
# Boolean literals
is_student = True
print(is_student) # Output:
True

has_passed = False print(has_passed)

# Output: False
```

1.8.4 Special Literal (None)

1.8.4 None

: None is a special literal representing the absence of a value.

Example:

```
# Special literal None result =
None print(result)
# Output: None
```

Note

- **Numeric Literals**: Numbers like integers, floats, and complex numbers.
- **String Literals**: Text data enclosed in quotes.
- **Boolean Literals**: True or False values.
- **Special Literal (**None**)**: Represents no value.

1.9 Data Types in Python

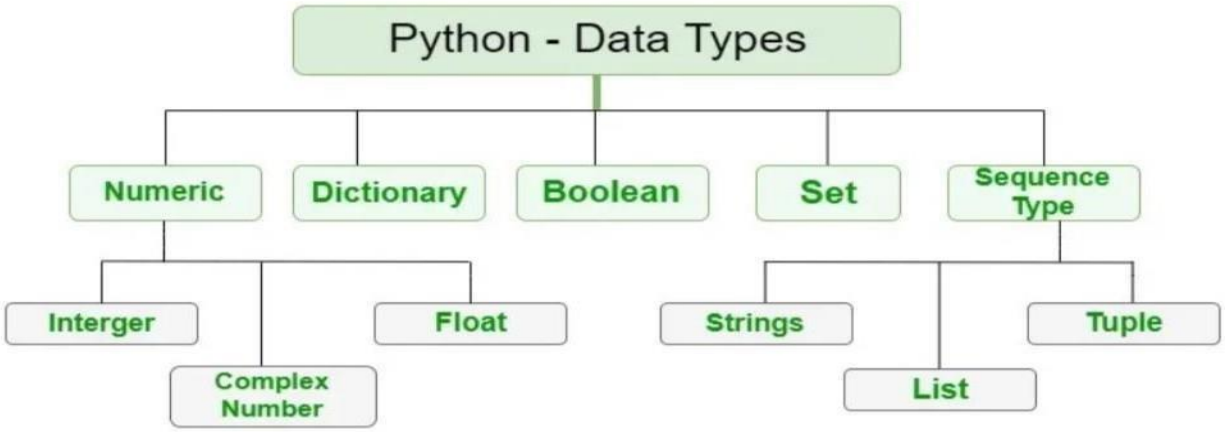
In Python, data types define the type of data a variable can hold. They determine what kind of operations can be performed on the data and how the data is stored in memory. Data types are classified into two types Immutable & Mutable data types.

Immutable Data Types: Immutable data types are those whose values cannot be changed after they are created.

Examples:
Strings: my_str = "hello" Tuples:
my_tuple = (1, 2, 3)
Numbers: my_int = 5, my_float = 5.0, my_complex = 5 + 3j Booleans: my_bool =
True

Mutable Data Types: Mutable data types are those whose values can be changed after they are created.

Examples:
Lists: my_list = [1, 2, 3] Dictionaries: my_dict = {'a':
1, 'b': 2} Sets: my_set = {1, 2, 3}



1.9.1 Numeric Types ; The numeric is immutable data type in Python represents the data that has a numeric value. A numeric value can be an integer, a floating number, or even a complex number.

1. **Integer Number** : This value is represented by int class. It contains positive or negative whole numbers (without fractions or decimals). In Python, there is no range for an integer value. Example:

```
a = 5 print(type(a))

#Output:
```

```
<class 'int'>
```

2. **Float Number**: This value is represented by the float class. It is a real number with a floating-point representation. It is specified by a decimal point.Example:

```
b = 5.0
print(type(b))

#Output:
<class 'float'>
```

3. Complex Number: A complex number is represented by a complex class. It is specified as (real part) + (imaginary part)j. Example:

```
c = 2 + 4j print(type(c))


#Output:
<class 'complex'>
```

1.9.2 Boolean Type :Boolean is an immutable data type with one of the two built-in values, True or False. Example:

```
print(type(True)) print(type(False))

#Output:
<class 'bool'>
<class 'bool'>
```

1.9.3 **Sequence Types**: Sequence types are used to store multiple items in an ordered manner.

1. **String type** : String is an immutable data type In Python, a string is a sequence of characters enclosed within single quotes (' '), double quotes (" "), triple single quotes (''' '''), or triple double quotes (""" """). Python does not have a separate character data type, so a single character is also treated as a string.Example:

```
s = 'Python' print(type(s))

Output:
<class 'str'>
```

2. **List type**: A list is a built-in, mutable data type in Python that holds an ordered collection of items. It can grow and shrink in size dynamically. Lists can store items of different types, including other lists.Example:

```
l = [10, 'b', 15.9]
print(type(l))

Output:
<class 'list'>
```

3. **Tuple type:** A tuple is an immutable, ordered collection of elements in Python. Once created, its elements cannot be changed. Tuples can contain elements of different data types.Example

```
t = (1, 2, 'd', 6.7)
print(type(t))
```

#The output will be:

<class 'tuple'>

1.9.4 Dictionary type: dictionary is an ordered collection of mutable data types. It stores values in key-value pairs, where both the keys and values can have different data types. The keys are immutable, and the values are mutable.

```
d = {1: "One", "two": 2, 3.0: [3, 4, 5]}
print(type(d)) output:
<class 'dict'>
```

1.9.5 Set Type: A Python set is a mutable, unordered collection of unique items that can have different data types.Example:

```
s = {1, "two", 3.0, (4, 5)}
print(type(s)) output:
<class 'set'>
```

1.10. String Operations , Methods & Functions:

Definition: A string in Python is a sequence of characters enclosed within single quotes ', double quotes ", or triple quotes '''or """. Strings are used to represent and manipulate text data.Strings in Python are immutable, meaning once a string is created, it cannot be changed.

1.10.1 String Operation:. String operations that work directly on string literals or variables using operators. These operations allow you to manipulate and work with strings in flexible and powerful ways. Various string operations are given in the table below. For example

```
s1 = "Python" s2 =
"Program" print(s1,
s2)
```

Output: Python Program

S.No	Operation	Syntax	Description	Example
1	Concatenation	str1 + str2	Combines two or more strings.	print(s1 + " " + s2) Output: Python Program
2	Repetition	str * n	Repeats a string multiple times.	print(s1 * 2) Output: Python Python
3	Slicing	str[start:end]	Extracts a substring from a string.	print(s1[1:4]) Output: yth
4	Indexing	str[index]	Accesses a specific character in a string.	print(s2[0]) Output: P
5	Membership	sub in str	Checks if a substring exists within another string.	print("tho" in s1) Output: True
6	Comparison	str1 == str2	Compares two strings for equality.	print(s1 == "Python") Output: True

1.10.2 String methods : String methods in Python are built-in functions that can be called on string objects to perform specific tasks. These methods provide a convenient way to manipulate and work with strings. Various string methods are given in the table below. For example

```
s = "Python"
print(s)
```

Output: "Python"

S.No	Method	Syntax	Description	Example
1	upper()	str.upper()	Converts all characters to uppercase.	print(s.upper()) Output: "PYTHON"
2	lower()	str.lower()	Converts all characters to lowercase.	print(s.lower()) Output: "python"
3	strip()	str.strip()	Removes leading and trailing whitespace.	print(s.strip()) Output: "Python"
4	replace(old, new)	str.replace(old, new)	Replaces occurrences of a substring with a new substring.	print(s.replace("Python", "Program")) Output: "Program"
5	find(sub)	str.find(sub)	Returns the lowest index of the substring if found.	print(s.find("y")) Output: 1
6	split(sep)	str.split(sep)	Splits the string into a list of substrings based on a separator.	print(s.split("t")) Output: ["Py", "hon"]
7	join(iterable)	str.join(iterable)	Joins elements of an iterable into a single string.	print(" ".join([s, "program"])) Output: "Python program"
8	title()	str.title()	Converts the first character of each word to uppercase.	print(s.title()) Output: "Python"
9	isalpha()	str.isalpha()	Checks if all characters are alphabetic.	print(s.isalpha()) Output: True
10	isdigit()	str.isdigit()	Checks if all characters are digits.	print(s.isdigit()) Output: False

1.10.3 String Functions: String functions in Python refer to built-in functions that operate on string objects, providing various ways to manipulate and handle string data. These functions are not called on string objects directly but can be applied to them. Various string functions are given in the table below.For example

S.No	Function	Syntax	Description	Example
1	len()	len(str)	Returns the length of the string.	print(len(s)) Output: 6
2	print()	print(str)	Prints the string to the console.	print(s) Output: Python
3	str()	str(obj)	Converts an object to a string representation.	print(str(123)) Output: "123"
4	format()	str.format(*args, **kwargs)	Formats the string using placeholders.	print("{} {}".format(s, "is fun")) Output: "Python is fun"

5	ord()	ord(char)	Returns the Unicode code point for a given character.	print(ord('P')) Output: 80
6	chr()	chr(code)	Returns the character that represents the given Unicode code point.	print(chr(80)) Output: 'P'
7	input()	input(prompt)	Reads a string from user input.	input("Enter something: ") Depends on user input
8	eval()	eval(expression)	Evaluates a string expression and returns the result.	print(eval("3 + 5")) Output: 8
9	repr()	repr(obj)	Returns a string representation of an object that can be used to recreate the object.	repr(s) print(repr(s)) Output: '"Python"'
10	ascii()	ascii(obj)	Returns a string representation of an object with non-ASCII characters escaped.	print(ascii("Python")) Output: 'Python'

1.11 List Operations, Methods & Functions

Definition: A list in Python is a collection of items, which can be of different types, enclosed within square brackets []. Lists are ordered, mutable, and can contain duplicate elements. Example

Creating lists

l1 = [1, 2, 3, 4, 5, 6, 7, 8]

l2 = ['Hello', 3.5, 'abc', 4]

1.11.1 List operations that work directly on list literals or variables using operators. These operations allow you to manipulate and work with lists in flexible ways. Various List operation are given in the table below.

S.No	Operation	Syntax	Definition	Example
1	Concatenation	list1 + list2	Concatenates two lists.	l1 + l2 Output: [1, 2, 3, 4, 5, 6, 7, 8, 'Hello', 3.5, 'abc', 4]
2	Repetition	list * n	Repeats the list n times.	l1 * 2 Output: [1, 2, 3, 4, 5, 6, 7, 8, 1, 2, 3, 4, 5, 6, 7, 8]
3	Access Element	list[index]	Accesses the element at the specified index.	print(l2[1]) Output: 3.5
4	List Length	len(list)	Returns the length of the list.	len(l1) Output: 8 len(l2) Output: 4
5	List Membership	element in list	Checks if the element is present in the list.	'Hello' in l2 Output: True

6	Slice from start	list[start:]	Returns a slice from the start index to the end of the list.	l1[1:] Output: ['2', '3', '4', '5', '6', '7', '8']
7	Slice to end	list[:stop]	Returns a slice from the beginning to the stop index.	l1[:3] Output: ['1', '2', '3']
8	Slice with step	list[start: stop: step]	Returns a slice from start to stop index with a step.	l1[::2] Output: ['1', '3', '5', '7']
9	Slice entire list	list[:]	Returns a copy of the entire list.	l1[:] Output: ['1', '2', '3', '4', '5', '6', '7', '8']
10	Negative indexing	list[-start:]	Returns a slice using negative indexing from the end.	l1[-3:] Output: ['6', '7', '8']
11	Negative step	list[::-1]	Reverses the list using slicing with a negative step.	l1[::-1] Output: ['8', '7', '6', '5', '4', '3', '2', '1']

1.11.2 List Methods List methods are built-in functions in Python that you can use to modify or gather information about lists. Various List Methods are given in the table below.

S.No	Method	Syntax	Description	Example
1	append()	list.append(element)	Adds an element to the end of the list.	l1.append(9) Output: [1, 2, 3, 4, 5, 6, 7, 8, 9]
2	extend()	list.extend(iterable)	Adds all elements of an iterable to the end of the list.	l1.extend(l2) Output: [1, 2, 3, 4, 5, 6, 7, 8, 'Hello', 3.5, 'abc', 4]
3	insert()	list.insert(index, element)	Inserts an element at the specified position.	l1.insert(1, 'a') Output: [1, 'a', 2, 3, 4, 5, 6, 7, 8]
4	remove()	list.remove(element)	Removes the first occurrence of the element.	l2.remove(4) Output: ['Hello', 3.5, 'abc']
5	pop()	list.pop(index)	Removes and returns the element at the specified position.	l1.pop(2) Output: 3 (list after pop: [1, 2, 4, 5, 6, 7, 8])
6	index()	list.index(element)	Returns the index of the first occurrence of the element.	l1.index(4) Output: 2
7	count()	list.count(element)	Returns the number of occurrences of the element.	l1.count(4) Output: 1
8	sort()	list.sort()	Sorts the list in ascending order.	l1.sort() Output: [1, 2, 3, 4, 5, 6, 7, 8]
9	reverse()	list.reverse()	Reverses the elements of the list.	l1.reverse() Output: [8, 7, 6, 5, 4, 3, 2, 1]
10	clear()	list.clear()	Removes all elements from the list.	l1.clear() Output: []

1.11.3 List Functions: List functions are built-in functions in Python that you can use to perform various actions on lists. These functions help you manipulate and analyze lists efficiently. Let us take same example:

S. No	Function	Syntax	Description	Example
1	len()	len(list)	Returns the number of elements in the list.	len(l1) Output: 8 len(l2) Output: 4
2	max()	max(list)	Returns the largest element in the list.	max(l1) Output: 8
3	min()	min(list)	Returns the smallest element in the list.	min(l1) Output: 1
4	sum()	sum(list)	Returns the sum of all elements in the list.	sum(l1) Output: 36
5	sorted()	sorted(list)	Returns a new sorted list from the elements in the list.	sorted(l1) Output: [1, 2, 3, 4, 5, 6, 7, 8]
6	list()	list(iterable)	Converts an iterable to a list.	list("hello") Output: ['h', 'e', 'l', 'l', 'o']
7	any()	any(iterable)	Returns True if any element of the iterable is true.	any([0, 1, 2]) Output: True
8	all()	all(iterable)	Returns True if all elements of the iterable are true.	all([1, 2, 3]) Output: True
9	enumerate()	enumerate(iterable)	Returns an enumerate object of (index, value) pairs.	list(enumerate(l2)) Output: [(0, 'Hello'), (1, 3.5), (2, 'abc'), (3, 4)]
10	zip()	zip(*iterables)	Aggregates elements from two or more iterables.	list(zip(l1, l2)) Output: [(1, 'Hello'), (2, 3.5), (3, 'abc'), (4, 4)]

1.12 Tuple operations, Methods & Functions:

Definition: A tuple in Python is a collection of items, which can be of different types, enclosed within parentheses (). Tuples are ordered and immutable, meaning that once they are created, their elements cannot be changed. Tuples can contain duplicate elements and can store elements of various data types.

Example

t1 = (1, 2, 3, 4, 5, 6, 7, 8)

t2 = ('Hello', 3.5, 'abc', 4)

1.12.1 Tuple operations: Tuple operations that work directly on tuple literals or variables using operators allow you to manipulate and work with tuples in flexible ways. Various tuple operations are given in the table below.

S.No	Operation	Syntax	Definition	Example
1	Concatenation	t1 + t2	Combines two tuples into a new one.	t1 + t2 Output: (1, 2, 3, 4, 5, 6, 7, 8, 'Hello', 3.5, 'abc', 4)
2	Repetition	t1 * 2	Repeats the elements of a tuple n times.	t1 * 2 Output: (1, 2, 3, 4, 5, 6, 7, 8, 1, 2, 3, 4, 5, 6, 7, 8)
3	Access Element	t1[0]	Accesses an element in a tuple by its index.	t1[0] Output: 1
4	Tuple Length	len(t1)	Returns the number of elements in the tuple.	len(t1) Output: 8
5	Tuple Membership	'Hello' in t2	Checks if an element exists within a tuple.	'Hello' in t2 Output: True
6	Slice from Start	t1[:4]	Extracts elements from the beginning to index 4.	t1[:4] Output: (1, 2, 3, 4)
7	Slice to End	t1[4:]	Extracts elements from index 4 to the end.	t1[4:] Output: (5, 6, 7, 8)
8	Slice with Step	t1[1:6:2]	Extracts elements from index 1 to 6 with step 2.	t1[1:6:2] Output: (2, 4, 6)
9	Slice Entire Tuple	t1[:]	Extracts all elements of the tuple.	t1[:] Output: (1, 2, 3, 4, 5, 6, 7, 8)
10	Negative Indexing	t1[-1]	Accesses an element using negative indexing.	t1[-1] Output: 8
11	Negative Step	t1[::-1]	Reverses the tuple using negative step.	t1[::-1] Output: (8, 7, 6, 5, 4, 3, 2, 1)

1.12.2 Tuple Methods : Tuple methods in Python are limited due to the immutability of tuples. These methods are built-in functions that can be called on tuple objects to perform specific tasks. Python has two built-in methods that you can use on tuples.

S.No	Method	Syntax	Description	Example
1	count()	tuple.count(x)	Returns the number of times x appears in the tuple.	t1.count(3) Output: 1
2	index()	tuple.index(x)	Returns the index of the first occurrence of x in the tuple.	t1.index(4) Output: 3

1.12.3 Tuple Functions: Tuple functions in Python refer to built-in functions that operate on tuple objects, providing various ways to manipulate and handle tuple data. Various tuple functions are given in the table below.

S.No	**Function**	**Syntax**	**Description**	**Example**
1	len()	len(tuple)	Returns the number of elements in the tuple.	len(t1) Output: 8
2	max()	max(tuple)	Returns the largest element in the tuple.	max(t1) Output: 8
3	min()	min(tuple)	Returns the smallest element in the tuple.	min(t1) Output: 1

4	sum()	sum(tuple)	Returns the sum of all elements in the tuple (numeric elements only).	sum(t1) Output: 36
5	sorted()	sorted(tuple)	Returns a sorted list of the tuple's elements.	sorted(t1) Output: [1, 2, 3, 4, 5, 6, 7, 8]
6	list()	list(tuple)	Converts the tuple into a list.	list(t1) Output: [1, 2, 3, 4, 5, 6, 7, 8]
9	enumerate()	enumerate(tuple)	Returns an enumerate object containing index and value pairs from the tuple.	list(enumerate(t1)) Output: [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 6), (6, 7), (7, 8)]
10	zip()	zip(tuple1, tuple2)	Aggregates elements from two tuples.	list(zip(t1, t2)) Output: [(1, 'Hello'), (2, 3.5), (3, 'abc'), (4, 4)]

1.13 Set operations, Methods & Functions:

Definition:A set is a collection of unique items, which means it cannot contain duplicate elements. Sets are unordered, so the items have no specific order, and they are mutable, meaning that their elements can be changed after creation. Sets are defined by placing items inside curly braces {} separated by commas, or by using the set() function.

Examples

s1 = {1, 2, 3, 4, 5, 6, 7, 8}

s2 = {'Hello', 3.5, 'abc', 4}

1.13.1 : Set operations: Set operations work directly on set literals or variables using operators to manipulate and work with sets in flexible ways. Various set operations are given in the table below.

S.No	Operation	Syntax	Definition	Example
1	Union	s1 | s2	Combines two sets into a new set with all unique elements.	s2 | s2 Output:{1, 2, 3, 4, 5, 6, 7, 8, 'Hello', 3.5, 'abc'}`
2	Intersection	s1 & s2	Returns a new set with elements common to both sets.	s1 & s2 Output: {4}
3	Difference	s1 - s2	Returns a new set with elements in the first set but not in the second.	s1 - s2 Output: {1, 2, 3, 5, 6, 7, 8}
4	Symmetric Difference	s1 ^ s2	Returns a new set with elements in either set but not both.	s1 ^ s2 Output: {1, 2, 3, 5, 6, 7, 8, 'Hello', 3.5, 'abc'}
5	Set Membership	x in s1	Checks if an element x is present in the set.	4 in s1 Output: True

1.13.2 Set Methods: Set methods in Python are built-in functions that can be called on set objects to perform specific tasks. Various set Methods are given in the table below

S.No	Method	Syntax	Description	Example
1	add	s1.add(x)	Adds an element to the set.	s1.add(9) Output: {1, 2, 3, 4, 5, 6, 7, 8, 9}
2	remove	s1.rm(x)	Removes an element. Raises Key Error if not found.	s1.rm(3) Output: {1, 2, 4, 5, 6, 7, 8}
3	discard	s1.disc(x)	Removes an element if present.	s1.disc(3) Output: {1, 2, 4, 5, 6, 7, 8}
4	pop	s1.pop()	Removes and returns an arbitrary element.	s1.pop() Output: 1 (remaining set: {2, 3, 4, 5, 6, 7, 8})
5	clear	s1.clr()	Removes all elements from the set.	s1.clr() Output: set()
6	update	s1.upd(t)	Adds all elements from t to s1.	s1.upd(s2) Output: {1, 2, 3, 4, 5, 6, 7, 8, 'Hello', 3.5, 'abc'}
7	Intersect upd	s1.int_upd(t)	Updates s1 with elements common to both sets.	s1.int_upd(s2) Output: {4}
8	diff_upd	s1.diff_upd(t)	Updates s1 by removing elements found in t.	s1.diff_upd(s2) Output: {1, 2, 3, 5, 6, 7, 8}
9	sym_diff_upd	s1.sym_upd(t)	Updates s1 with elements in either set but not both.	s1.sym_upd(s2) Output: {1, 2, 3, 5, 6, 7, 8, 'Hello', 3.5, 'abc'}
10	issubset	s1.issub(t)	Returns True if all elements of s1 are in t.	s1.issub(s2) Output: False

1.13.3 Set Functions: Set functions in Python refer to built-in functions that operate on set objects, providing various ways to manipulate and handle set data. Various set Functions are given in the table below

S.No	**Function**	**Syntax**	**Description**	**Example**
1	len()	len(s1)	Returns the number of elements in the set.	len(s1) Output: 8
2	max()	max(s1)	Returns the largest element in the set.	max(s1) Output: 8
3	min()	min(s1)	Returns the smallest element in the set.	min(s1) Output: 1
4	sum()	sum(s1)	Returns the sum of all elements in the set.	sum(s1) Output: 36
5	sorted()	sorted(s1)	Returns a sorted list of the set's elements.	sorted(s1) Output: [1, 2, 3, 4, 5, 6, 7, 8]
6	any()	any(s1)	Returns True if any element in the set is true.	any(s1) Output: True
7	all()	all(s1)	Returns True if all elements in the set are true.	all(s1) Output: True
8	enumerate()	enumerate(s1)	Returns an enumerate object.	list(enumerate(s1)) Output: [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 6), (6, 7), (7, 8)]
9	set()	set(iterable)	Converts an iterable into a set.	set([1, 2, 3]) Output: {1, 2, 3}
10	frozen set()	frozen set(s1)	Returns an immutable frozen set from the set.	frozen set(s1) Output: frozen set({1, 2, 3, 4, 5, 6, 7, 8})

1.14 Dictionary Operations, Methods & Functions

Definition: A dictionary is a mutable data type; however, its keys are immutable. A dictionary in Python is a collection of key-value pairs, where each key is associated with a value. Values can be of any type and are mutable. Dictionaries are unordered and are defined by placing key-value pairs inside curly braces {} separated by commas, with a colon: separating keys and values. Example

d = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
d1 = {'e': 5, 'f': 6}

1.14.1 Dictionary operations: Dictionary operations work directly on dictionary literals or variables using operators to manipulate and work with dictionaries in flexible ways. Various dictionary operations are given in the table below.

S.No	Operation	Syntax	Description	Example
1	Accessing Values	d[key]	Accesses the value associated with the given key.	d['a'] Output: 1
2	Update Value	d[key] = value	Modifies an existing entry or adds a new one.	d['a'] = 10 Output: {'a': 10, 'b': 2, 'c': 3, 'd': 4}
3	Adding Values	d[key] = value	Adds a new key-value pair to the dictionary.	d['g'] = 7 Output: {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'g': 7}
4	Deleting Key	del d[key]	Deletes the key-value pair from the dictionary.	del d['a'] Output: {'b': 2, 'c': 3, 'd': 4}
5	Clearing Dictionary	d.clear()	Removes all elements from the dictionary.	d.clear() Output: {}
6	Deleting Dictionary	del d	Deletes the entire dictionary.	del d
7	Copying Dictionary	d.copy()	Returns a shallow copy of the dictionary.	d_copy = d.copy() Output: {'a': 1, 'b': 2, 'c': 3, 'd': 4}
8	Checking Key Existence	key in d	Checks if the key exists in the dictionary.	'a' in d Output: True
9	Iterating Keys	for key in d	Iterates over the keys in the dictionary.	for key in d Output: 'a', 'b', 'c', 'd'

1.14.2 Dictionary Methods: Dictionary methods in Python are built-in functions that can be called on dictionary objects to perform specific tasks. Various dictionary Methods are given in the table below.

S.No	Method	Syntax	Description	Example
1	clear	d.clr()	Removes all elements from the dictionary.	d.clr() Output: {}
2	copy	d.cp()	Returns a shallow copy of the dictionary.	d_cp = d.cp() Output: {'a': 1, 'b': 2, 'c': 3, 'd': 4}
3	fromkeys	dict.fk(seq,v)	Creates a new dictionary with keys from seq and values set to v.	dict.fk(['a', 'b', 'c'], 0) Output: {'a': 0, 'b': 0, 'c': 0}

4	get	d.gt(k)	Returns the value for key if key is in dictionary, else default.	d.gt('a') Output: 1
5	items	d.it()	Returns a view object of dictionary's key-value pairs.	d.it() Output: dict_items([('a', 1), ('b', 2), ('c', 3), ('d', 4)])
6	keys	d.ks()	Returns a view object of dictionary's keys.	d.ks() Output: dict_keys(['a', 'b', 'c', 'd'])
7	pop	d.pp(k)	Removes the specified key and returns the corresponding value.	d.pp('a') Output: 1 (remaining: {'b': 2, 'c': 3, 'd': 4})
8	popitem	d.ppit()	Removes and returns an arbitrary (key, value) pair.	d.ppit() Output: ('d', 4) (remaining: {'a': 1, 'b': 2, 'c': 3})
9	setdefault	d.sd(k, v)	Returns the value of key if it is in the dictionary; if not, inserts key with a value of default and returns default.	d.sd('e', 5) Output: 5 (if e does not exist)
10	update	d.up(d1)	Updates the dictionary with the key-value pairs from other.	d.up(d1) Output: {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5, 'f': 6}
11	values	d.vl()	Returns a view object of dictionary's values.	d.vl() Output: dict_values([1, 2, 3, 4])

1.14.3 Dictionary Functions: Dictionary functions in Python refer to built-in functions that operate on dictionary objects, providing various ways to manipulate and handle dictionary data. Various dictionary Functions are given in the table below

S.No	Function	Syntax	Description	Example
1	len()	len(d)	Returns the number of key-value pairs in the dictionary.	len(d) Output: 4
2	max()	max(d)	Returns the maximum key in the dictionary.	max(d) Output: 'd'
3	min()	min(d)	Returns the minimum key in the dictionary.	min(d) Output: 'a'
4	sorted()	sorted(d)	Returns a sorted list of the dictionary's keys.	sorted(d) Output: ['a', 'b', 'c', 'd']
5	any()	any(d)	Returns True if any key in the dictionary is true.	any(d) Output: True
6	all()	all(d)	Returns True if all keys in the dictionary are true.	all(d) Output: True
7	enumerate()	enumerate(d)	Returns an enumerate object of the dictionary's keys.	list(enumerate(d)) Output: [(0, 'a'), (1, 'b'), (2, 'c'), (3, 'd')]
8	zip()	zip(d, d1)	Returns an iterator of tuples from the dictionaries' keys.	list(zip(d, d1)) Output: [('a', 'e'), ('b', 'f')]
9	reversed()	reversed(d)	Returns a reversed iterator over the keys of the dictionary.	list(reversed(d)) Output: ['d', 'c', 'b', 'a']

10	dict()	dict(d1)	Converts a list of tuples into a dictionary.	dict([('e', 5), ('f', 6)]) Output: {'e': 5, 'f': 6}
11	str()	str(d)	Converts the dictionary to a string representation.	str(d) Output: "{'a': 1, 'b': 2, 'c': 3, 'd': 4}"
12	type()	type(d)	Returns the type of the dictionary object.	type(d) Output: <class 'dict'>

SUMMARY :String,List,Set,Tuple,Dictionary

Feature	String	List	Set	Tuple	Dictionary
1.Definition	Ordered collection of characters.	Ordered collection of items.	Unordered collection of unique items.	Ordered collection of items, immutable.	Unordered collection of key-value pairs.
2.Mutability	Immutable	Mutable	Mutable	Immutable	Mutable
3.Syntax	“str”or ‘str’	[1, 2, 3]	{1, 2, 3}	(1, 2, 3)	{‘key’: ‘value’}
4.Indexing	Supports indexing	Supports indexing	Does not support indexing	Supports indexing	Keys can be used as indices
5.Duplication	Allows duplicates	Allows duplicates	Does not allow duplicates	Allows duplicates	Keys must be unique
6.Order	Maintains order	Maintains order	Does not maintain order	Maintains order	Does not maintain order
7.Methods	.upper(), .find(), .replace(), etc.	.append(), .extend(), .remove(), etc.	.add(), .remove(), .union(), etc.	No specific methods for mutation.	.get(), .keys(), .values(), etc.
8.Applications	Text processing, data representation.	Collections of related items, dynamic arrays.	Mathematical sets, eliminating duplicates.	Fixed collections of items, read-only sequences.	Databases, configurations, associative arrays.
9.Performance	Efficient for small text data.	Efficient for small to moderate collections.	Efficient for membership tests.	Faster than lists for fixed-size collections.	Efficient for lookups, especially when keys are integers or strings.
10.Use Case	"Hello, World!"	[1, 'two', 3.0]	{1, 2, 3}	(1, 'two', 3.0)	{'name': 'Alice', 'age': 30}

1.15 Operators: Operators are special symbols that operators on operands. operands may be variable or value. The operators +, -, *, / and ** perform addition, subtraction, multiplication, division and exponentiation.

Various types of operators in python are

- Arithmetic operators
- Comparison (Relational) operators
- Logical (Boolean) operators
- Bitwise operators
- Assignment operators
- Membership and Identity Operators

1.15.1 Arithmetic Operators: Arithmetic operators are used to perform mathematical operations such as addition, subtraction, multiplication, and division. Example

x = 15
y = 4

S.No	Operator	Meaning	Example
1	+	Add two operands or unary plus	print('x + y =', x + y) Output: x + y = 19
2	-	Subtract right operand from the left or unary minus	print('x - y =', x - y) Output: x - y = 11
3	*	Multiply two operands	print('x * y =', x * y) Output: x * y = 60
4	/	Divide left operand by the right one (results in a float)	print('x / y =', x / y) Output: x / y = 3.75
5	%	Modulus - remainder of the division of left operand by the right	print('x % y =', x % y) Output: x % y = 3

6	//	Floor division - results in a whole number adjusted left	print('x // y =', x // y)
			Output: x // y = 3

7	**	Exponent - left operand raised to the power of right	print('x ** y =', x ** y) Output: x ** y = 50625

1.15.2 Comparison Operators: Comparison operators or relational operator compare values and return a Boolean result (True or False). Example

x = 10
y = 12

S.No	Operator	Meaning	Example
1	>	Greater than - True if left operand is greater than the right	print(x > y) Output: False
2	<	Less than - True if left operand is less than the right	print(x < y) Output: True
3	==	Equal to - True if both operands are equal	print(x == y) Output: False
4	!=	Not equal to - True if operands are not equal	print(x != y) Output: True
5	>=	Greater than or equal to - True if left operand is greater than or equal to the right	print(x >= y) Output: False
6	<=	Less than or equal to - True if left operand is less than or equal to the right	print(x <= y) Output: True

1.15.3 Logical Operators: Logical operators are used to combine conditional statements. Example

x = True
y = False

S.No	Operator	Meaning	Example
1	and	True if both the operands are true	print('x and y is', x and y) Output: x and y is False
2	or	True if either of the operands is true	print('x or y is', x or y) Output: x or y is True
3	not	True if operand is false (complements the operand)	print('not x is', not x) Output: not x is False

1.15.4 Bitwise Operators: Bitwise operators act on binary numbers, performing bit-by-bit

operations. x = 10 # (0000 1010 in binary)
y = 4 # (0000 0100 in binary)

S.No	Operator	Meaning	Example
1	&	Bitwise AND	print('x & y =', x & y) Output: x & y = 0
3	~	Bitwise NOT	print('~x =', ~x) Output: ~x = -11
4	^	Bitwise XOR	print('x ^ y =', x ^ y) Output: x ^ y = 14
5	>>	Bitwise right shift	print('x >> 2 =', x >> 2) Output: x >> 2 = 2
6	<<	Bitwise left shift	print('x << 2 =', x << 2) Output: x << 2 = 40

1.15.5 Assignment Operators: Assignment operators are used to assign values to variables. Example

S.No	Operator	Meaning	Code Example	Output
1	=	Assign value	x = 5 print('x =', x)	x = 5
2	+=	Add and assign	x = 10 x += 5 print('x =', x)	x = 15
3	-=	Subtract and assign	x = 15 x -= 5 print('x =', x)	x = 10

4	*=	Multiply and assign	x = 5 x *= 5 print('x =', x)	x = 25
5	/=	Divide and assign	x = 25 x /= 5	x = 5
			print('x =', x)	
6	%=	Modulus and assign	x = 8 x %= 5 print('x =', x)	x = 3
7	//=	Floor divide and assign	x = 25 x //= 5 print('x =', x)	x = 5
8	**=	Exponentiate and assign	x = 2 x **= 5 print('x =', x)	x = 32
9	&=	Bitwise AND and assign	x = 14 x &= 5 print('x =', x)	x = 4
10	\=	Bitwise OR and assign	x = 10 `x \= 5 print('x =', x)`	X = 5
11	^=	Bitwise XOR and assign	x = 10 x ^= 5 print('x =', x)	x = 15
12	>>=	Bitwise right shift and assign	x = 160 x >>= 5 print('x =', x)	x = 5
13	<<=	Bitwise left shift and assign	x = 5 x <<= 5 print('x =', x)	x = 160
14	(Tuple)	Tuple assignment	t = (4, 5, 6) x, y, z = t print(x, y, z)	4 5 6
15	(Multiple)	Multiple assignment	a, b, c = 1, 2, 3 print(a, b, c)	1 2 3

1.15.6 Membership Operators: special operators include identity operators (isand is not) and membership operators (inand not in). Membership Operators are used to test whether a value or variable is found in a sequence (string, list, tuple, set, and dictionary). In a dictionary, we can only test for the presence of a key, not the value.

S.No	Operator	Meaning	Code Example	Output
1	in	True if value/variable is found in the sequence	x = 'Hello world' print ('world' in x)	True
2	not in	True if value/variable is not found in the sequence	y = {1: 'a', 2: 'b'} print (3 not in y)	True

1.15.7 Identity Operators: Identity Operators are used to check if two values (or variables) are located on the same part of the memory. Two variables that are equal do not imply that they are identical.

S.No	Operator	Meaning	Code Example	Output
1	is	True if the operands are identical (refer to the same object)	x1 = 5 y1 = 5 print(x1 is y1)	True
2	is not	True if the operands are not identical (do not refer to the same object)	x3 = [1, 2, 3] y3 = [1, 2, 3] print(x3 is not y3)	True

1.16 Operator precedence & Associativity

In Python, **operator precedence** determines the order in which operators are evaluated in expressions. Operators with higher precedence are evaluated before those with lower precedence. Here's a simple example to illustrate operator precedence. **Associativity** determines the order in which operators of the same precedence are evaluated. Operators can be left-associative or right-associative.

Operator Precedence and Associativity

S.No	Operator	Description	Associativity
1	()	Parentheses	Left-to-right
2	**	Exponentiation	Right-to-left
3	+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT	Right-to-left
4	*, /, //, %	Multiplication, Division, Floor division, Modulus	Left-to-right
5	+, -	Addition, Subtraction	Left-to-right
6	<<, >>	Bitwise shift operators	Left-to-right
7	&	Bitwise AND	Left-to-right
8	^	Bitwise XOR	Left-to-right
9	||	Bitwise OR	Left-to-right
10	==, !=, >, <, >=, <=	Comparison operators	Left-to-right
11	is, is not	Identity operators	Left-to-right
12	in, not in	Membership operators	Left-to-right
13	not	Logical NOT	Right-to-left
14	and	Logical AND	Left-to-right
15	or	Logical OR	Left-to-right

Python's operator precedence and associativity support the PEMDAS rule. Let's see an example

```
result = 2 + 3 * (4 ** 2) // 8 - 1
```

1. **Parentheses (P):** Evaluate the expression inside the parentheses first.
   - (4∗∗2)(4 ** 2) -> 16
   - Result so far: 2+3∗16//8−12 + 3 * 16 // 8 - 1
2. **Exponentiation (E):** Perform the exponentiation inside the parentheses.
   - 4∗∗24 ** 2 -> 16
   - Result so far: 2+3∗16//8−12 + 3 * 16 // 8 - 1
3. **Multiplication (M):** Perform multiplication.
   - 3∗163 * 16 -> 48
   - Result so far: 2+48//8−12 + 48 // 8 - 1
4. **Division (D):** Perform division.
   - 48//848 // 8 -> 6
   - Result so far: 2+6−12 + 6 - 1
5. **Addition (A):** Perform addition.
   - 2+62 + 6 -> 8
   - Result so far: 8−18 - 1
6. **Subtraction (S):** Perform subtraction.
   - 8−18 - 1 -> 7
   - Final result: 77

```
print(result)  # Output: 7
```

1.17 Conditional and Control Constructs.:

Conditional Constructs are programming structures, also known as conditional statements, that allow your code to make decisions. They execute certain blocks of code based on whether specific conditions are true or false. This enables different paths of execution within a program.

Control Constructs: Control Constructs are programming structures that manage the flow of execution within a program. They include both conditional constructs and looping constructs, allowing you to make decisions, repeat actions, and jump to different parts of the code based on specific conditions or criteria.

Classification:

1. **Conditional Constructs (Control Constructs):**
    a. **If Statements:** Executes code if a condition is true.
    b. **If-Else Statements:** Executes one block of code if a condition is true, and another if it is false.
    c. **Elif (Else If) Statements:** Checks multiple conditions in sequence, executing the code for the first true condition.
    d. **Nested If Statements:** If statements inside other if statements to check multiple conditions.
    e. **Ternary Conditional Operator:** A concise way to make a conditional assignment in one line.
2. **Looping Constructs (Control Constructs):**
    a. **For Loop:** Iterates over a sequence (e.g., list, range).
    b. **While Loop:** Repeats a block of code as long as a condition is true.
3. **Jump Statements (Control Constructs):**
    a. **Break:** Exits a loop prematurely.
    b. **Continue:** Skips the current iteration and proceeds to the next iteration of the loop.
    c. **Pass:** Acts as a placeholder, doing nothing when executed.

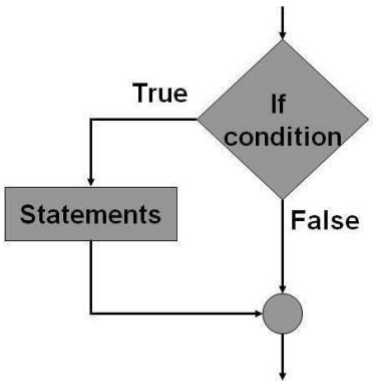
1.17.1 Conditional Constructs (Control Constructs):

a) ifStatement: The ifstatement checks a condition and executes a block of code if the condition is true.

Syntax:

```
if condition:
    # code block

# Example Greatest of Two numbers:

a = 15
b = 10

if a > b:
    print("a is greater")

#Output
a  is greater
```

Control flow diagram

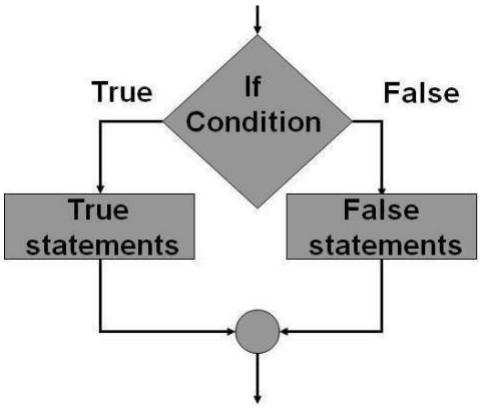


b) if. ElseStatement : The if. Else statement checks a condition, executes a block of code if the condition is true, and another block of code if the condition is false.

Syntax:

```
if condition:
        # code block if true else:
        # code block if false
```

Control flow diagram



#Example (Greatest of two numbers):

```
a = 10
b = 20

if a > b:
        print("a is greater") else:
        print("b is greater")
```

#Output

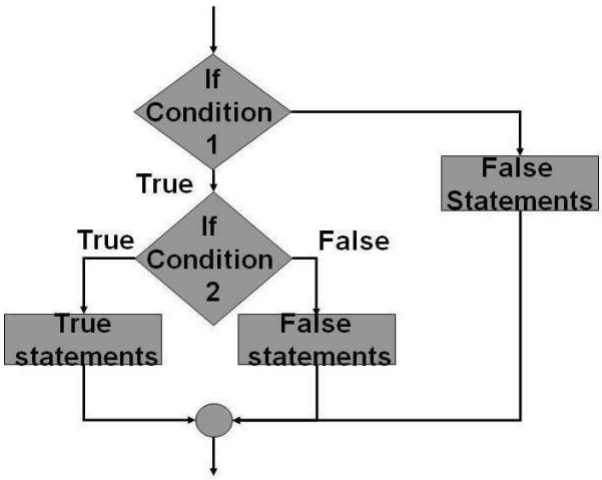
b is greater

C)if..elif..else Statement: The if. Elif. Else statement checks multiple conditions, and executes the corresponding code block for the first condition that is true.

Syntax:

```
if condition1:
        # code block if condition1 is true Elif condition2:
        # code block if condition2 is true else:
        # code block if none are true #Example (Greatest of
```

Control Flow diagram



three numbers):

```
a = 10
b = 20
c = 30

if a > b and a > c:
        print("a is the greatest") elif b > a and b >
c:
        print("b is the greatest") else:
        print("c is the greatest") #Outpt:
```

c is the greatest

d) Nested if Statement: The nested if statement allows you to place one if statement inside another if statement, enabling more complex condition checking.

Syntax:

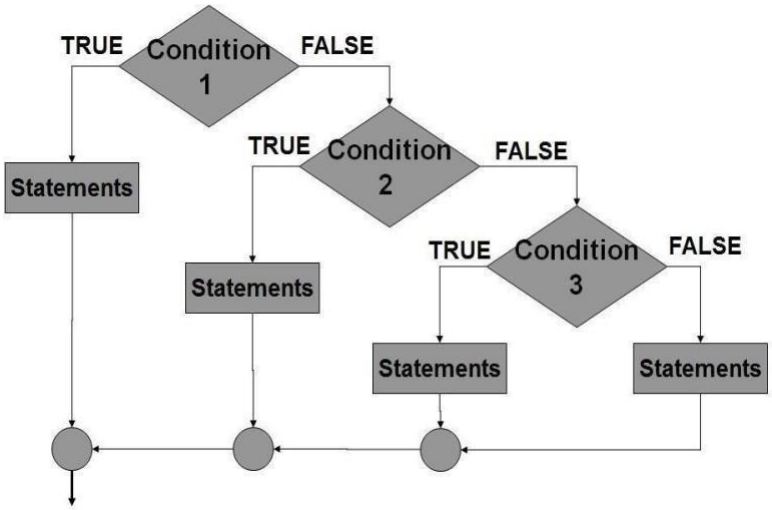
```
if condition1:
    if condition2:
        # code block if true else:
        # code block if false
else:
    # code block if false #Example Greatest
```

of 3 numbers

```
a = 10
b = 20
c = 30
if a >= b:
    if a >= c:
        print("a is the greatest")
    else:
        print("c is the greatest")
else:
    if b >= c:
        print("b is the greatest")
    else:
        print("c is the greatest")
```

#Output:
c is the greatest

Control Flow diagram



e) Ternary conditional operator: The ternary conditional operator also known as a conditional expression, allows you to choose between two values in a single line of code based on a condition.

Syntax:
value = true_value if condition else false_value

```
# greater of two numbers:
a = 10
b = 20

# Using ternary conditional operator to find the greater value greater_value = a if a
> b else b

print("The greater value is:", greater_value)
```

#Output:
The greater value is: 20

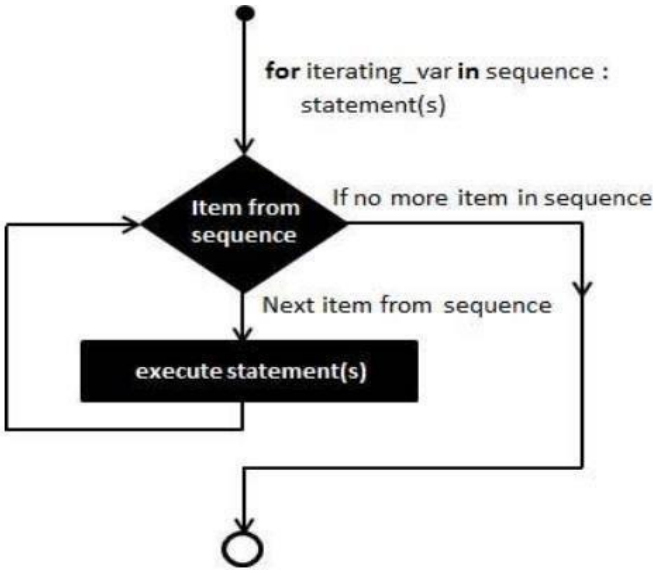
1.18 Looping Constructs (Control Constructs): **Looping Constructs** are programming structures that let you run a block of code repeatedly until the condition becomes false.

1.18.1 For Loop: A for loop repeatedly runs a block of code for each item in a sequence, like a list, tuple, string, or range. It processes each item one by one. Here are some variants of the for loop in Python. Message Copilot

Syntax
for variable in sequence: # code

to execute

Control Flow diagram



a) For Loop with Range: Executes a block of code a fixed number of times, iterating over a sequence of numbers.

Syntax:
for variable in range(start, end, step): # code to execute

Example:
```
# Using range(0, 10, 1)
for i in range(0, 10, 1): print(i, end=' ')

#Output:
0 1 2 3 4 5 6 7 8 9
```

b) For Loop with List: Iterates over each element in a list.

```
#Example:
numbers = [1, 2, 3, 4, 5] for num in
numbers:
        print(num)
```

c) For Loop with Tuple: Iterates over each element in a tuple.

```
#Example:
fruits = ('apple', 'banana', 'cherry') for fruit in fruits:
        print(fruit) #Output

apple
banana
cherry
```

d) For Loop with String: Iterates over each character in a string.

```
# Example
text = "hello" for char in
text:
        print(char, end="")  # Use 'end' to avoid new line after each character
```

Output:
hello

e) For Loop with Dictionary: Iterates over each key-value pair in a dictionary.

Example:
```
d = {'a': 1, 'b': 2, 'c': 3}
for key, value in d.items(): print(f"{key}:
          {value}")
```

```
#Output a:
1
b: 2
c: 3
```

f) Nested For Loop: A for loop inside another for loop.

Example:
```
for i in range(3):  # Outer loop for j in range(2):  #
        Inner loop
                print(f"i: {i}, j: {j}") #Output:
```

```
i: 0, j: 0
i: 0, j: 1
i: 1, j: 0
i: 1, j: 1
i: 2, j: 0
i: 2, j: 1
```

1.18.2 While Loop: A while loop repeatedly executes a block of code as long as a specified condition is true.

a) Simple whileLoop : A whileloop will execute a block of code as long as its condition is True

Syntax:

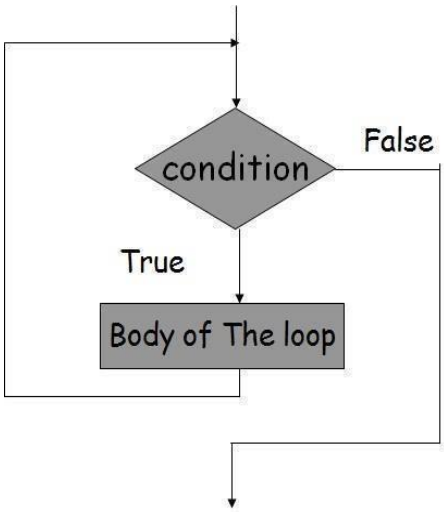
```
while condition:
        # code to execute
```

```
#Example Simple while loop count = 0
while count < 5: print(count, end=' ')
        count += 1
```

#Output:

0 1 2 3 4

Control Flow diagram



b) **While Loop with else:** A while loop can be combined with an else block. The else block will be executed when the while loop condition becomes False.

```
count = 0
while count < 5:
        print(count, end=" ")  # Use 'end' to print on the same line count += 1
else:
        print("Loop finished")  # This will also print on the same line, following the loop
```

Output:
0 1 2 3 4 Loop finished

By using the end=" " parameter in the print function, we ensure that the loop prints each number on the same line

C)Nested while loop : A nested while loop is when one loop runs inside another loop, and it can be particularly useful for handling multidimensional data

Syntax:

```
while outer_condition: while
        inner_condition:
                # code to execute
# Example Code:
i = 0
while i < 3:   # Outer loop j = 0
        while j < 2:   # Inner loop print(f"i: {i}, j:
                {j}") j += 1
        i += 1
```

Output:
i: 0, j: 0
i: 0, j: 1
i: 1, j: 0
i: 1, j: 1
i: 2, j: 0
i: 2, j: 1

1.17.3 Jump Statement In Python, a jump statement is used to control the flow of execution within loops or conditional statements, allowing you to exit from or skip parts of the code under certain conditions. The main jump statements in Python are:

a) Break: The break statement is used to exit a loop prematurely when a certain condition, specified with an if statement, is true. This allows you to stop the loop from further execution and continue with the code that follows after the loop.
Syntax : break

i) Break in for loop: Using break in a for loop saves time and resources by exiting early when a condition is met, avoiding unnecessary iterations. Control flow

```
for var in sequence:
    # codes inside for loop
    if  condition:
        break
    # codes inside for loop

# codes outside for loop
```

```
#Example Break with For loop:
for i in range(10): if i == 5:
        break   # Exits the loop when i is 5
    print(i, end='\t')  # Use end='\t' to print horizontally with a tab space
```

Output:
0 1 2 3 4

ii) Break in While loop: The break statement in a while loop can simulate a do-while loop by ensuring the loop body executes at least once before checking the condition. When the condition is met, the break statement exits the loop.Control flow

```
while test expression:
    # codes inside while loop
    if  condition:
        break
    # codes inside while loop

# codes outside while loop
```

```
#Example – Break with While count = 0
while True:
    print(count, end=" ")  # Print horizontally count += 1
    if count >= 5:
        break   # Exit the loop when count is greater than or equal to 5
```

Output:
0 1 2 3 4

b) Continue: The continue statement is used to skip the current iteration of the loop and immediately proceed to the next iteration. This is useful when you want to avoid executing certain code for specific conditions within the loop.

Syntax: continue

i) Continue in for loop:

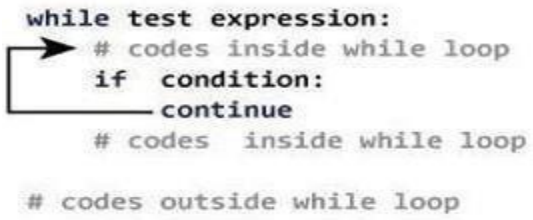
```
for var in sequence:
    # codes inside for loop
    if  condition:
        continue
    # codes inside for loop

# codes outside for loop
```

```
#Example for loop with continue for num in
range(10):
        if num % 2 == 0:
                continue  # Skip even numbers
        print(num, end=" ")  # Print odd numbers horizontally
```

Output:

1 3 5 7 9

ii) continue in while loop:




```
#Example-continue with while loop

count = 0
while count < 10: count
        += 1
        if count % 2 == 0:
                continue  # Skip even numbers
        print(count, end=" ")  # Print odd numbers horizontally
```

Output:

1 3 5 7 9

C) Pass: The pass statement is a null operation that acts as a placeholder. When executed, it does nothing and is often used to create minimal code blocks or during development as a placeholder for future code.

Syntax: pass

(i) Pass in for loop:

```
for i in range(10): if i == 5:
                pass  # Does nothing when i is 5 print(i, end=' ')

#Output
0 1 2 3 4 5 6 7 8 9
```

(ii) Pass in While loop:

```
i = 0
while i < 10:
        if i == 5:
                pass  # Does nothing when i is 5 print(i, end=' ')
        i += 1

#Output
0 1 2 3 4 5 6 7 8 9
```

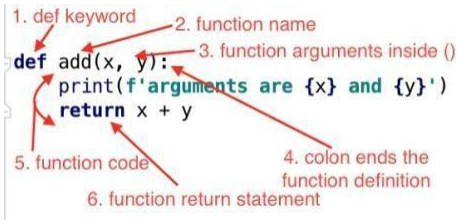
UNIT-II

PYTHON FUNCTIONS AND MODULES

Concept of Functions – Built-in Functions - Lambda Function – Recursive Function – Global and Local Variables – Decorators - “args” and “kwargs” - Iterators – Concept of Shallow and Deep Copy – map, reduce and filter methods - Context Manager – Comprehension -Basics of Built-in Modules – “sys” Module – Array Module – Math Module – “collections”and “itertools” Modules.

2.1 Concept of Functions:

In Python, a function is a block of code that can be reused to perform a specific task. Functions are self-contained modules that can be called when needed.



1. **Types of Functions in Python**

- **Built-in library function**: These are Standard functions in Python that are available to use.
- **User-defined function**: We can create our own functions based on our requirements.
- **Lambda Functions** :Lambda functions are small, anonymous functions defined with the lambda keyword.
- **Recursive Functions**:These are functions that call themselves in order to solve smaller instances of the same problem

Creating a Function:

In Python a function is defined using the def keyword.

Example

```
def my_function():
  print("Hello from a function")
```

Calling a Function:

To call a function, use the function name followed by parenthesis.

Example:

```
def my_function ():
```

```
    print("Hello from a function")

my_function()
```

Python Function Arguments:

Arguments are the values passed inside the parenthesis of the function. A function can have any number of arguments separated by a comma.

```
# A simple Python function to check whether x is even or odd

def evenOdd(x):

    if (x % 2 == 0):

        print("even")

    else:

        print("odd")

evenOdd(2)

evenOdd(3)
```

Output

```
even

odd
```

Benefits of Using Functions:

- Increase Code Readability
- Increase Code Reusability
- Modularity
- Maintainability

2.2 Built-in Functions

- Built-in functions in Python are pre-defined functions provided by the Python language that can be used to perform common tasks.
- To use a built-in function, simply call it with the appropriate arguments, like this **function_name(argument1, argument2)**

Python has a set of built-in functions.

1. Input/Output Functions
    - print() Displays output to the console
    - input() Takes user input as a string
    - format() Formats string

2. Type Conversion Functions

- int() Converts to an integer
- float() Converts to a float
- str() Converts to a string
- bool() Converts to a boolean
- list(), tuple(), set(), dict() Convert between data types

3. Mathematical Functions

- abs(x) Returns absolute value of x
- pow(x, y) Returns x raised to the power y (x^y)
- round(x, n) Rounds x to n decimal places
- min() / max() Returns the smallest/largest value

4. Sequence Functions

- len() Returns length of an object
- sorted() Returns a sorted list
- sum() Returns the sum of elements
- all() / any() Checks if all/any elements are True

5. String Functions

- ord(c) Returns ASCII value of c
- chr(n) Returns character from ASCII value n
- repr(x) Returns a string representation of x

6. Object and Variable Functions

- type(obj) Returns type of object
- id(obj) Returns memory address of object
- is instance(obj, type) Checks if an object is of a given type

7. File Handling Functions

- open(filename, mode) Opens a file in a specified mode
- read() / write() Reads/writes data to a file
- close() Closes a file

8. Functional Programming Functions

- map(func, iterable) Applies func to each item in iterable
- filter(func, iterable) Filters elements based on func
- zip() Combines multiple iterables
- lambda Creates an anonymous function

9. Memory Management Functions

- del Deletes a variable or object
- gc.collect() Forces garbage collection

2.3 Lambda Function:

- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments, but can only have one expression.

Syntax : lambda arguments : expression lambda:

The keyword to define the function.

arguments: A comma-separated list of input parameters (like in a regular function).
expression: A single expression that is evaluated and returned.

Example

Add 10 to argument a, and return the result:

```
x = lambda a : a + 10
print(x(5))
```

Example:

Multiply argument a with argument b and return the result:

```
x = lambda a, b : a * b

print(x(5, 6))
```

2. Lambda Functions with map():Used to apply a function to all items in an iterable (e.g., list, tuple).

Example:

```
numbers = [1, 2, 3, 4, 5]
squared = map(lambda x: x ** 2, numbers)
print(list(squared))  # Output: [1, 4, 9, 16, 25]
```

3. Lambda Functions with filter():Used to filter items in an iterable based on a condition.

Example:

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = filter(lambda x: x % 2 == 0, numbers)
print(list(even_numbers))  # Output: [2, 4, 6]
```

4. Lambda Functions with reduce(): Used to apply a function cumulatively to the items of an iterable, reducing them to a single value. (Note: reduce() is part of the functools module).

Example:

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)
print(product)  # Output: 120
```

5. Lambda Functions with Default Arguments:Used to create functions with default argument values.

Example:

```
full_name = lambda first, last="Doe": f"{first} {last}"
print(full_name("John"))  # Output: John Doe
```

Limitations:

Single Expression: Lambda functions can only have a single expression. They are not suitable for complex operations.

•**Readability**: Overusing lambda functions can sometimes make the code harder to read and understand, especially for more complex operations.

2.4Recursive Function

- Recursion involves a function calling itself directly or indirectly to solve a problem by breaking it down into simpler and more manageable parts.
- Recursive function is defined like any other function, but it includes a call to itself.

Example of Recursion

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n-1)
print(factorial(5))
```

output:120

2.5 Global and Local Variables :

2.5.1 Global variables are those which are not defined inside any function and have a global scope.

2.5.2 local variables are those which are defined inside a function and their scope is limited to that function only

Local Variables

2.5.3 Local variables in Python are those which are initialized inside a function and belong only to that particular function. It cannot be accessed anywhere outside the function

```
def my_function():
    x = 10  # Local variable
    print(x)
my_function()  # This will print 10
```

Global Variables

These are those which are defined outside any function and which are accessible throughout the program, i.e., inside and outside of every function.

```
x = 5  # Global variable
def my_function():
    print(x)  # Accessing global variable inside a function
my_function()  # This will print 5
print(x)  # This will also print 5, because x is global
```

The global Keyword

when you create a variable inside a function, that variable is local, and can only be used inside that function.To create a global variable inside a function, you can use the global keyword.

Example

If you use the global keyword, the variable belongs to the global scope:

```
def myfunc():
    global x
x = "fantastic" myfunc()
print("Python is " + x)
```

Comparison Basis	Global Variable	Local Variable
Definition	declared outside the functions	declared within the functions
Lifetime	They are created the execution of the program begins and are lost when the program is ended	They are created when the function starts its execution and are lost when the function ends
Data Sharing	Offers Data Sharing	It doesn't offers Data Sharing
Scope	Can be access throughout the code	Can access only inside the function
Parameters needed	parameter passing is not necessary	parameter passing is necessary
Storage	A fixed location selected by the compiler	They are kept on the stack
Value	Once the value changes it is reflected throughout the code	once changed the variable don't affect other functions of the program

2.6 Decorators:

- Decorators are a powerful and flexible way to modify or extend the behavior of functions or methods, without changing their actual code.
- A decorator is essentially a function that takes another function as an argument and returns a new function with enhanced functionality.

```
# A simple decorator function
def decorator(func):
    def wrapper():
        print("Before calling the function.")
        func()
        print("After calling the function.")
    return wrapper
# Applying the decorator to a function
@decorator
def greet():
    print("Hello, World!")
greet()
```

Output

```
Before calling the function.
Hello, World!
After calling the function.
```

Types of Decorators:

1. Function Decorators:

The most common type of decorator, which takes a function as input and returns a new function.

```
def simple_decorator(func):
    def wrapper():
        print("Before calling the function.")
```

```
        func()
        print("After calling the function.")
    return wrapper
@simple_decorator def
greet():
    print("Hello, World!")
greet()
```

Output

Before calling the function.

Hello, World!

After calling the function.

2.Method Decorators:

Used to decorate methods within a class. They often handle special cases, such as the self argument for instance methods.

```
def method_decorator(func):
    def wrapper(self, *args, **kwargs):
        print("Before method execution")
        res = func(self, *args, **kwargs)
        print("After method execution")
        return res
    return wrapper
class MyClass:
    @method_decorator
    def say_hello(self):
        print("Hello!")
obj = MyClass()
obj.say_hello()
```

Output

Before method execution Hello!

After method execution

3.Class Decorators

Class decorators are used to modify or enhance the behavior of a class. Like function decorators, class decorators are applied to the class definition. They work by taking the class as an argument and returning a modified version of the class.

```
def fun(cls):
    cls.class_name = cls.__name____
    return cls
@fun
class Person:
    pass
print(Person.class_name)
```

Output

```
Person
```

Common Use Cases of Decorators:

Logging: A decorator can be used to log function calls.

Authentication/Authorization: To check if a user has the proper rights before executing a function.

Memoization: Store results of expensive function calls to optimize performance by caching results.

Performance Measurement: Measure how long a function takes to execute.

Built-in Decorators:

Python provides some built-in decorators like @staticmethod, @classmethod, and @property:

- **@staticmethod**: Used to define a method that doesn't operate on an instance of the class.
- **@classmethod:** Used to define a method that operates on the class itself, rather than an instance.

- **@property**: Used to define a method that behaves like an attribute.

```
class MyClass:
    @staticmethod
    def static_method():
        print("This is a static method.")
    @classmethod
    def class_method(cls):
        print(f"This is a class method. cls: {cls}")
    @property
    def my_property(self):
return "This is a property."
    obj= MyClass()
obj.static_method()
obj.class_method()
print(obj.my_property)
```

output:

This is a static method.

This is a class method. cls: <class '_main_.MyClass'> This is

a property.

2.7 “args” and “kwargs”

*args and **kwargs are used to pass a variable number of arguments to a function. They allow you to call a function with any number of positional and keyword arguments, making your code more flexible and reusable.

Special Symbols Used for passing arguments in Python:

- *args (Non-Keyword Arguments)
- **kwargs (Keyword Arguments)

***args (Positional Arguments)**

*args is used to pass a variable number of positional arguments to a function. The *args syntax collects extra positional arguments as a tuple.

Example:

```
def my_function(*args):
```

```
    for arg in args:
        print(arg)
my_function(1, 2, 3, 4)
```

output:

1

2

3

4

****kwargs (Keyword Arguments)**

**kwargs is used to pass a variable number of keyword arguments (arguments passed by name) to a function. The **kwargs syntax collects these arguments into a dictionary.

Example:

```
def my_function(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")
my_function(name="Alice", age=25)
```

output:

name: Alice age:

25

Combining *args and **kwargs

You can use both *args and **kwargs in the same function, but *args must appear before **kwargs.

Example:

```
def my_function(a, b, *args, **kwargs):
    print(f"a: {a}, b: {b}")
    print(f"args: {args}")
    print(f"kwargs: {kwargs}")
my_function(1, 2, 3, 4, 5, name="Alice", age=25)
```

output:

a: 1, b: 2

```
args: (3, 4, 5)
kwargs: {'name': 'Alice', 'age': 25}
```

Why Use *args and **kwargs?

- **Flexibility**: You can write functions that can accept any number of arguments, which is helpful in many cases like function wrappers (decorators), API endpoints, and more.
- **Code Readability**: Using *args and **kwargs makes it clear that a function is meant to handle flexible arguments, rather than enforcing a strict number of arguments

2.8 ITERATOR

- An iterator is an object that holds a sequence of values and provide sequential traversal through a collection of items such as lists, tuples and dictionaries. .
- The Python iterators object is initialized using the **iter()** method.
- It uses the **next()** method for iteration.

Creating an Iterator

An iterator is an object that follows the iterator protocol, which consists of two methods:

- __iter__() → Returns the iterator object itself.
- _next_() → Returns the next item from the sequence, and when it reaches the end, it raises a Stop Iteration exception.

Example:

```
class MyIterator:
    def _init_(self, start, end):
        self.current = start
        self.end = end
    def _iter_(self):
        return self
    def __next__(self):
        if self.current >= self.end:
            raise Stop Iteration # No more items
        else:
            self.current += 1
            return self.current - 1
```

```
# Using the custom iterator
iterator = MyIterator(1, 5) for
num in iterator:
    print(num)
```

output:

1

2

3

4

Built-in Iterators

Python provides several built-in objects that are iterable, meaning you can use them in a forloop or with next(). Examples include lists, tuples, strings, dictionaries, and more.

Example with a List:

```
my_list = [10, 20, 30]
# Get an iterator object from the list iterator
= iter(my_list)
# Using next() to iterate
print(next(iterator))  # 10
print(next(iterator))  # 20
print(next(iterator))  # 30
# print(next(iterator))  # Uncommenting this will raise StopIteration Output:
```

10

20

30

2.9 Shallow copy

- A shallow copy creates a new object but retains references to the objects contained within the original. It only copies the top-level structure without duplicating nested elements.

- Changes made to a copy of an object do reflect in the original object. In python, this is implemented using the **"copy.copy()"** function.

Syntax of Python Shallowcopy

```
copy.copy()
```

Example:

```
import copy
# Original list with nested lists
original = [[1, 2], [3, 4]]
# Shallow copy
shallow_copy = copy.copy(original)

# Modify a nested object in the shallow copy
shallow_copy[0][0] = 99

print(original)        # Output: [[99, 2], [3, 4]] (original is affected)
print(shallow_copy)    # Output: [[99, 2], [3, 4]] (shallow_copy is also affected)
```

Deep copy in Python

- A deep copy creates a new compound object before inserting copies of the items found in the original into it in a recursive manner.
- It will first construct a new collection object and then recursively populate it with copies of the child objects found in the original. It means that any changes made to a copy of the object do not reflect in the original object.

Syntax of Python Deepcopy

```
copy.deepcopy()
```

Example:

```
import copy

# Original list with nested lists
original = [[1, 2], [3, 4]]

# Deep copy
deep_copy = copy.deepcopy(original)

# Modify a nested object in the deep copy
deep_copy[0][0] = 99

print(original)        # Output: [[1, 2], [3, 4]] (original is not affected)
print(deep_copy)       # Output: [[99, 2], [3, 4]] (deep_copy is modified)
```

Differences Between Shallow and Deep Copy

Shallow Copy	Deep Copy
• Only the outer object is copied. Nested objects are shared between the original and the copy.	• Both the outer object and all nested objects are copied.
• Modifying a nested object in the copy affects the original (and vice versa).	• Modifying the nested objects in the copy does not affect the original.
• More memory efficient because it shares inner objects.	• More memory consuming as it creates copies of all nested objects.
• When you don't need to independently modify nested objects.	• When you want completely independent objects.

2.10 map, reduce and filter methods.

In Python, map(), reduce(), and filter() are functional programming tools that allow you to work with sequences (like lists or tuples) in a more efficient and compact way. These functions provide a more declarative approach to processing collections of data.

1. map()

The map() function applies a given function to all items in an iterable (like a list or a tuple) and returns a map object (an iterator) which can be converted to other data types like a list or tuple.

Syntax:

map(fun, iter)

fun: It is a function to which map passes each element of given iterabl

iter: iterable object to be mapped.

Example

```
# Function to return double of n def
double(n):
        return n * 2
# Using map to double all numbers numbers
= [5, 6, 7, 8]
result = map(double, numbers)
print(list(result))
```

Output

```
[10, 12, 14, 16]
```

reduce()

The reduce function is used to apply a particular function passed in its argument to all of the list elements mentioned in the sequence passed along.This function is defined in “functools” module.

Syntax: reduce(func, iterable[, initial])

Parameters:

- **fun**: It is a function to execuate on each element of the iterable objec
- **iter**: It is iterable to be reduced

Example:

```
import functools
# Define a list of numbers
numbers = [1, 2, 3, 4]
# Use reduce to compute the product of list elements product
= functools.reduce(lambda x, y: x * y, numbers)
print("Product of list elements:", product)
```

Output

```
Product of list elements: 24
```

filter()

The filter() method filters the given sequence with the help of a function that tests each element in the sequence to be true or not.

Syntax: filter(function, sequence)

- **function:** function that tests if each element of a sequence is true or not.
- **sequence:** sequence which needs to be filtered, it can be sets, lists, tuples, or containers of any iterators.

Example:

```
# Define a function to check if a number is even def

is_even(n):

    return n % 2 == 0

# Define a list of numbers

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Use filter to filter out even numbers

even_numbers = filter(is_even, numbers)

print("Even numbers:", list(even_numbers))
```

Output

Even numbers: [2, 4, 6, 8, 10]

2.10 Context Manager:

A **context manager** is a type of object that defines a runtime context for a block of code. It allows you to allocate and release resources precisely when you want to. Context managers are commonly used in scenarios like working with files, network connections, database connections, or acquiring and releasing locks.

A context manager is defined by two methods:

1. **__enter____()**: This method is executed when the withblock is entered. It is used to set up any necessary resources or actions.
2. **__exit____()**: This method is executed when the withblock is exited (either normally or due to an exception). It is used for cleanup, such as closing files, releasing locks, or handling exceptions.

Example

 # Python program creating a context manager

```
class ContextManager():
        def __init__(self):
                print('init method called')
        def _enter_(self):
                print('enter method called')
                return self
        def _exit_(self, exc_type, exc_value, exc_traceback):
                print('exit method called')
    with ContextManager() as manager:
        print('with statement block')
```

Output:

```
init method called
enter method called
with statement block
exit method called
```

2.11 Comprehension:

Comprehensions provide us with a short and concise way to construct new sequences (such as lists, sets, dictionaries, etc.) using previously defined sequences. Python supports the following 4 types of comprehension:

- List Comprehensions
- Dictionary Comprehensions
- Set Comprehensions
- Generator Comprehensions

List Comprehensions

List Comprehensions provide an elegant way to create new lists. The following is the basic structure of list comprehension:

Syntax: output_list = [output_exp for var in input_list if (var satisfies this condition)]

Example : Generating Even list using List comprehensions

```
input_list = [1, 2, 3, 4, 4, 5, 6, 7, 7]
list_using_comp = [var for var in input_list if var % 2 == 0] print("Output
List using list comprehensions:",list_using_comp)
```

Output:

```
List using list comprehensions: [2, 4, 4, 6]
```

Dictionary Comprehensions

Extending the idea of list comprehensions, we can also create a dictionary using dictionary comprehensions. The basic structure of a dictionary comprehension looks like below.

output_dict = {key:value for (key, value) in iterable if (key, value satisfy this condition)}

Example : Mapping states with their capitals with using dictionary comprehension state

= ['Gujarat', 'Maharashtra', 'Rajasthan']

capital = ['Gandhinagar', 'Mumbai', 'Jaipur']

dict_using_comp = {key:value for (key, value) in zip(state, capital)} print("Output

Dictionary using dictionary comprehensions:", dict_using_comp) Output:

Dictionary using dictionary comprehensions: {'Rajasthan': 'Jaipur',

'Maharashtra': 'Mumbai',

'Gujarat': 'Gandhinagar'}

Set Comprehensions

Set comprehensions are pretty similar to list comprehensions. The only difference between them is that set comprehensions use curly brackets { }

Example : Checking Even number using set comprehension

input_list = [1, 2, 3, 4, 4, 5, 6, 6, 6, 7, 7]

set_using_comp = {var for var in input_list if var % 2 == 0}

print("Output Set using set comprehensions:",set_using_comp)

Output:

Set using set comprehensions: {2, 4, 6}

Generator Comprehensions

Generator Comprehensions are very similar to list comprehensions. One difference between them is that generator comprehensions use circular brackets whereas list comprehensions use square brackets.

Example:

```
input_list = [1, 2, 3, 4, 4, 5, 6, 7, 7]
output_gen = (var for var in input_list if var % 2 == 0) print("Output
values using generator comprehensions:", end = ' ') for var in
output_gen:
                print(var, end = ' ')
```

Output:

Output values using generator comprehensions: 2 4 4 6

2.12 Basics of Built-in Modules:

A built-in module is a file containing Python code that provides functionality for a specific task. Built-in modules are pre-installed modules in Python that you can import and use in your programs. These modules provide reusable functions and classes to make development faster and easier.

How to Use a Built-in Module?
To use a built-in module, you simply import it using the import statement.

For example:

```
import math
print(math.sqrt(16)) #
Output: 4.0
```

Commonly Used Built-in Modules

1. math

The math module provides mathematical functions like trigonometric operations, logarithms, and more.

```
import math
print(math.pi)  # Output: 3.141592653589793
```

2. datetime

The datetime module provides classes for manipulating dates and times. import datetime

```
now = datetime.datetime.now()
print(now)  # Output: Current date and time
```

3. Os

The os module provides a way to interact with the operating system, including tasks like navigating the file system, working with environment variables, and handling paths.

```
import os
print(os.getcwd())  # Output: Current working directory
```

4. random

The random module is used to generate random numbers and make random selections.

```
import random
print(random.randint(1, 10))  # Output: Random integer between 1 and 10
```

5. *sys*

The sys module provides access to system-specific parameters and functions. It's useful for working with the interpreter, command-line arguments, and environment.

```
import sys
```

```
print(sys.argv)  # Output: Command line arguments passed to the script
```

6. ***re (Regular Expressions)***

The re module allows you to work with regular expressions to search, match, and manipulate strings.

```
import  re patter=
r'\d+'
text = "There are 42 apples." match
= re.findall(pattern, text)
print(match) # Output: ['42']
```

7. json

The json module helps you work with JSON (JavaScript Object Notation) data, which is often used for data exchange in APIs and web services.

```
import json
data = {"name": "Alice", "age": 30}
json_string = json.dumps(data)
print(json_string)  # Output: '{"name": "Alice", "age": 30}'
```

8. collections

The collections module provides alternatives to built-in types, such as namedtuples, deque, Counter, and more.

9. ***itertools***

The itertools module provides functions that create iterators for efficient looping, like combinations, permutations, and more.

10. ***time***

The time module provides functions for working with time, such as sleeping, measuring time intervals, and formatting time.

```
import time
time.sleep(2)  # Pauses execution for 2 seconds
```

11. ***functools***

The functools module provides higher-order functions to work with functions themselves, like memoization, partial functions, etc.

2.13 "sys" Module:

The sys module in Python provides access to system-specific parameters and functions. It allows interaction with the interpreter, handling command-line arguments, controlling input/output, and more.

Commonly Used Features of sys Module:

1. Command-Line Arguments (sys.argv)

- sys.argv is a list containing command-line arguments passed to the script.

  sys.argv[0] is the script name, and sys.argv[1:] are additional arguments.

  Example:

```
import sys
print("Script Name:", sys.argv[0])
print("Arguments:", sys.argv[1:])
```

2. Exit a Program (sys.exit())

- Used to terminate a script with an optional exit status.
- sys.exit(0) indicates a successful termination.
- sys.exit(1) or any non-zero value indicates an error.

```
import sys
sys.exit("Exiting the program")
```

3. ***Standard Input, Output, and Error (sys.stdin, sys.stdout, sys.stderr)***

- Redirect input/output streams

```
import sys
sys.stdout.write("Hello, World!\n")
sys.stderr.write("This is an error message\n")
```

2. Python Version Information (sys.version & sys.version_info)

- Get the version of Python running the script.

```
import sys
print(sys.version)
print(sys.version_info)  # More structured version info
```

3. System Path (sys.path)

- List of directories searched for module imports.

```
import sys
print(sys.path)  # Shows the list of paths Python searches for modules
```

4. Check Platform (sys.platform)

- Identifies the OS (e.g., 'win32' for Windows, 'linux' for Linux, 'darwin' for macOS). import sys

```
print(sys.platform)
```

5. Get System Recursion Limit (sys.getrecursionlimit())

- Retrieves or modifies the recursion limit.

```
import sys
print("Recursion Limit:", sys.getrecursionlimit())
sys.setrecursionlimit(2000) # Change recursion depth
```

6. Get Python Interpreter Path (sys.executable)

- Returns the path of the Python interpreter being used.

```
import sys
print(sys.executable)
```

7. Get System Max Size (sys.maxsize)

- Returns the largest integer a variable can hold

```
import sys
print(sys.maxsize)  # Useful for handling large data structures
```

8. Garbage Collection (sys.getrefcount())

- Shows the reference count of an object.

```
import sys
x = []
print(sys.getrefcount(x))  # Returns reference count (typically +1 due to function argument passing)
```

2.14 Array module

The array module in Python provides an efficient way to store and manipulate arrays of numeric values. Unlike Python lists, which can store different types of objects, arrays in the array module are more memory-efficient because they store elements of the same type.

Importing the Array Module

To use arrays, you need to import the module:

```
import array
```

Creating an Array

Array in Python can be created by importing an array module. array(data_type , value_list) is used to create array in Python with data type and value list specified in its arguments.

You can create an arr import array #

Creating an integer array

```
arr = array.array('i', [1, 2, 3, 4, 5])

print(arr)  # Output: array('i', [1, 2, 3, 4, 5]) ay using:
```

- **typecode**: Defines the type of elements stored (e.g., 'i'for integers, 'f'for floats).
- **initializer**: An optional list of values.

Common Array Operations:

1. **Appending Elements**

```
arr.append(6)

print(arr)  # Output: array('i', [1, 2, 3, 4, 5, 6])
```

2. ***Inserting Elements***

```
arr.insert(2, 10)  # Insert 10 at index 2

print(arr)  # Output: array('i', [1, 2, 10, 3, 4, 5, 6])
```

3. ***Removing Elements***

```
arr.remove(10) # Removes first occurrence of 10

print(arr)  # Output: array('i', [1, 2, 3, 4, 5, 6])
```

4. ***Popping an Element***

arr.pop(3) # Removes element at index 3 print(arr)

Output: array('i', [1, 2, 3, 5, 6])

5. ***Finding Index of an Element***

print(arr.index(5)) # Output: 3

6. ***Reversing an Array***

arr.reverse()

print(arr) # Output: array('i', [6, 5, 3, 2, 1])

7. ***Converting Array to List***

lst = arr.tolist()

print(lst) # Output: [6, 5, 3, 2, 1]

Advantages of array Module

- More memory-efficient than lists for large datasets.
- Faster performance for numerical operations compared to lists.

2.15 Math Module

The math module in Python provides mathematical functions for performing complex calculations, including trigonometry, logarithms, factorials, and more.

Importing the math Module

import math

Common Functions in math Module:

1. **Basic Mathematical Function**

print(math.sqrt(16)) # Square root → 4.0

print(math.pow(2, 3)) # Power (2^3) → 8.0

print(math.factorial(5)) # Factorial (5!) → 120

print(math.fabs(-10)) # Absolute value → 10.0

```
print(math.gcd(48, 18))  # Greatest Common Divisor (GCD) → 6
```

2. **Rounding and Precision**

```
print(math.floor(3.9)) # Round down → 3
print(math.ceil(3.1))  # Round up → 4
print(math.trunc(4.9))  # Truncate decimal → 4
```

3. **Trigonometric Functions**

```
print(math.sin(math.pi/2)) # Sin(90°) → 1.0
print(math.cos(0))         # Cos(0°) → 1.0
print(math.tan(math.pi/4)) # Tan(45°) → 1.0
```

4. ***Logarithmic Functions***

```
print(math.log(10))        # Natural log (base e) → 2.302
print(math.log10(100))     # Log base 10 → 2.0
print(math.exp(2))         # e^2 → 7.389
```

5. ***Constants in math Module***

```
print(math.pi)     # Pi → 3.141592653589793
print(math.e)      # Euler's number → 2.718281828459045
print(math.tau)  # Tau (2π) → 6.283185307179586
print(math.inf)  # Infinity → inf
print(-math.inf)  # Negative Infinity → -inf
```

6. ***Degree-Radian Conversion***

```
print(math.degrees(math.pi))  # Convert radians to degrees → 180.0
print(math.radians(180))       # Convert degrees to radians → 3.141592653589793
```

When to Use math Module?

- When precision in mathematical computations is needed.
- For scientific and engineering calculations.

- When working with trigonometry or logarithmic functions.

2.16 “collections”and “itertools” Modules.

In Python, the collections and itertools modules offer specialized tools for handling data structures and iterators, respectively.

collections Module:

The collections module provides alternatives to Python's built-in data types, offering specialized container datatypes. Key components include:

namedtuple(): Factory function for creating tuple subclasses with named fields.

from collections import namedtuple

Define a named tuple called 'Point'

Point = namedtuple('Point', ['x', 'y']) #

Create an instance of Point

p = Point(10, 20) print(p.x)

Output: 10 print(p.y)

Output: 20

deque: List-like container with fast appends and pops on either end.

from collections import deque

Create a deque with some initial values

d = deque([1, 2, 3, 4, 5])

Add elements to the right end d.append(6)

Add elements to the left end

d.appendleft(0)

print(d) # Output: deque([0, 1, 2, 3, 4, 5, 6]) #

Remove elements from the right end d.pop()

Remove elements from the left end

d.popleft()

print(d) # Output: deque([1, 2, 3, 4, 5])

ChainMap: Dict-like class for creating a single view of multiple mappings.

Counter: Dict subclass for counting hashable objects.

from collections import Counter

Count the occurrences of each character in the string c =

Counter('hello world')

print(c) # Output: Counter({'l': 3, 'o': 2, 'h': 1, 'e': 1, ' ': 1, 'w': 1, 'r': 1, 'd': 1}) #

Most common elements

print(c.most_common(2)) # Output: [('l', 3), ('o', 2)]

OrderedDict: Dict subclass that remembers the order entries were added. from

collections import OrderedDict

Create an OrderedDict and add some key-value pairs

Od = OrderedDict()

od['one'] = 1

od['two'] = 2

od['three'] = 3

print(od) # Output: OrderedDict([('one', 1), ('two', 2), ('three', 3)]) #

Iterate over the OrderedDict

for key, value in od.items():

print(key, value)

Output: #

one 1

two 2

three 3

defaultdict: Dict subclass that calls a factory function to supply missing values. from

collections import defaultdict

Create a defaultdict with list as the default factory

dd = defaultdict(list)

Add some key-value pairs

dd['a'].append(1)

dd['a'].append(2)

dd['b'].append(3)

print(dd) # Output: defaultdict(<class 'list'>, {'a': [1, 2], 'b': [3]})

UserDict, UserList, UserString: Wrappers around dictionary, list, and string objects for easier subclassing.

itertools Module:

The itertools module offers a collection of fast, memory-efficient tools for creating and working with iterators. Notable functions include:

count(): Returns an iterator that generates consecutive integers, starting from a specified number.

import itertools

for number in itertools.count(10, 2): # Starts at 10, increments by 2

print(number)

if number >= 20:

break

cycle(): Returns an iterator that repeats the input iterable indefinitely. re

import itertools

counter = 0

for item in itertools.cycle(['A', 'B', 'C']):

```
        print(item)

        counter += 1

        if counter == 6:
```

repeat(item, times=None)

- Repeats an item for a specified number of times (times). If times is not specified, it repeats the item indefinitely.

```
import itertools

for x in itertools.repeat(5, 3):  # Repeats 5 three times

    print(x)
```

chain():

- Combines multiple iterables into a single iterable.

```
import itertools

for item in itertools.chain('ABC','DEF'):

    print(item)
```

zip_longest():

- Combines multiple iterables into tuples, filling missing values with a specified fillvalue.

```
import itertools

for pair in itertools.zip_longest('AB', '123', fillvalue='0'):

    print(pair)
```

combinations():

- Returns all possible combinations of a specified length from the input iterable.

```
import itertools

items = ['A', 'B', 'C', 'D']

for combination in itertools.combinations(items, 2):
```

```
print(combination)
```

permutations():

- Returns all possible permutations of a specified length from the input iterble. import itertools

```
items = ['A', 'B', 'C']
for perm in
    itertools.permutations(items,
    2): print(perm)
```

UNIT – III
PYTHON OBJECT ORIENTED CONCEPTS

Class and Object – constructor – Destructor – Inheritance – Abstract Class – Data Hiding – “super”and “class” Methods – Dunder Method – Static and Instance Methods – “self” Parameter – Developing and Deploying Packages.

3.1 Class and Object 3.1.1Class

A class is like a blueprint or template for creating objects (instances).
It defines the attributes and methods that the objects created from the class will have.

Attributes : are variables that belong to the class or object.

Methods : are functions defined inside the class that describe the behaviors of the objects.

1. Class Definition:

In Python, a class is defined using the class keyword. It's a template that defines attributes and methods .

Example of class:

```
class ClassName:
    # Constructor
    def __init__(self, parameters):
        # Initialize the object's attributes
        self.attribute1 = value1
        self.attribute2 = value2
    # Method
    def method_name(self):
        # Method logic
        pass
```

2. Constructor (__init__):

The __init__method is a special method in Python classes, called the constructor.
It gets automatically called when you create a new object of the class.
The purpose of the constructor is to initialize the object's state (i.e., its attributes).

3. Self:

The self keyword is a reference to the current instance of the class.
It allows you to access attributes and methods of the class from within its own methods. While calling methods from outside the class, you do not need to pass self — Python handles it automatically.

4. Attributes:

Attributes are variables associated with a class and its instances. There are two types:

Instance Attributes: These are attributes that belong to the instance of the class.

Class Attributes: These are attributes that are shared by all instances of the class.

5. Methods:

Methods are functions that are defined inside a class. They define the behaviours of the objects created from the class.

Methods can access and modify instance attributes and perform actions based on them.

6. Inheritance:

Python allows you to create a new class from an existing class, a feature known as inheritance.

The new class inherits all attributes and methods from the parent class, but you can also add or modify its own attributes and methods.

7. Encapsulation:

Encapsulation refers to bundling the data (attributes) and methods that operate on the data into a single unit (the class), and restricting access to some of the object's components. This is typically done by making some attributes private.

In Python, private attributes and methods are indicated by a leading double underscore (__).

8. Polymorphism:

Polymorphism allows different classes to have methods with the same name, but their behaviours may differ. This is useful in situations where different classes share a common interface but have their own implementations.

3.1.2Object

An object is an instance of a class. It represents an actual entity that you can work with, created from the blueprint defined by the class.

Here's how you can create objects from the Dog class:

```
dog1 = Dog("Buddy", 3)
dog2 = Dog("Lucy", 5)
print(dog1.name) # Output: Buddy
print(dog2.age)  # Output: 5
print(dog1.bark())  # Output: Buddy says woof!
```

Example of class and object:

```
class Dog:
    def _init_(self, name, breed):
        self.name = name
        self.breed = breed
    def bark(self):
     print(f"{self.name} is barking!")  dog1
= Dog("Buddy", "Golden Retriever") dog2
= Dog("Max", "Bulldog") print(dog1.name)
# Output: Buddy dog1.bark()  # Output:
Buddy is barking! print(dog2.name)  #
Output: Max dog2.bark()  # Output: Max is
barking!
```

Python Class Essentials

- A 'class' in Python is a blueprint for creating objects, encapsulating data and methods in a structured way.
- To define a class, use the class keyword followed by the class name and a colon.
- Attributes and methods are defined within the class to establish its functionality.

- Instantiating objects from the class is done using its name and parentheses, e.g., p1 = MyClass().
- Accessing attributes is straightforward with dot notation, like p1.x to output 5.
- Special methods like __init___act as constructors to initialize objects' states.
- The self parameter in methods refers to the instance itself.

3.2 Constructor and destructor

In Python, constructors and destructors are special methods used to initialize and clean up objects when they are created and destroyed, respectively.

1. Constructor (__init___method):

The constructor is a special method used to initialize newly created objects. It is automatically called when a new instance of the class is created. The constructor's main role is to assign initial values to the object's attributes.

Syntax: The constructor is always defined using the __init____method in Python.
It takes at least one argument, self, which refers to the current instance of the class. The constructor can have additional parameters to initialize attributes when an object is created.

Example of constructor

```
class Person:
    def _init_(self, name, age): # Constructor
        self.name = name
        self.age = age
    def greet(self):
     print(f"Hello, my name is {self.name} and I am {self.age} years old.") person1
= Person("Alice", 30)
print(person1.name) # Output: Alice
print(person1.age)  # Output: 30
person1.greet()        # Output: Hello, my name is Alice and I am 30 years old.
```

2. Destructor (__del___method):

- The destructor is a special method in Python, defined using the _del___method. It is called when an object is about to be destroyed (i.e., when it is no longer in use and gets garbage collected). The destructor is typically used to clean up resources, such as closing files or network connections, before the object is deleted.
- The destructor method is rarely needed in Python because Python uses automatic garbage collection, but it can still be defined if you need to perform some cleanup.
- The destructor method takes only one argument, self, which refers to the current instance of the object.

Example of Destructor:

```
class MyClass:
    def _init_(self, value):
        self.value = value
        print(f"Object with value {self.value} created.")
```

```
    def __del__(self):
        print(f"Object with value {self.value} is being deleted.")
obj = MyClass(10)
del obj  # Output: Object with value 10 is being deleted.
```

When is the Destructor Called?

- The destructor is called when an object is deleted using the del statement or when an object goes out of scope.
- In a program with circular references, the destructor may not be called immediately because Python's garbage collection may not remove objects with circular references until a later time.

3.3 Inheritance in python

Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a class to inherit attributes and methods from another class. In Python, inheritance allows you to create a new class (subclass) based on an existing class (superclass). This promotes code reuse and enables you to extend and modify the behaviour of the parent class.

Types of Inheritance in Python:

Single Inheritance: A subclass inherits from one parent class.

Multiple Inheritance: A subclass inherits from more than one parent class.

Multilevel Inheritance: A subclass inherits from another subclass

Hierarchical Inheritance: Multiple subclasses inherit from the same parent class.

Hybrid Inheritance: A combination of multiple types of inheritance.

Syntax of Inheritance:

```
class ParentClass:
    pass
class ChildClass(ParentClass):
    pass
```

Single Inheritance Example:

- In single inheritance, a subclass inherits from a single parent class.

```
class Animal:
    def _init_(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} makes a sound.") class
Dog(Animal):
    def __init__(self, name, breed):
        super()._init_(name) # Call the parent class constructor
        self.breed = breed
    def speak(self):
        print(f"{self.name} barks.")
dog = Dog("Buddy", "Golden Retriever")
print(dog.name)  # Output: Buddy
print(dog.breed) # Output: Golden Retriever
```

```
dog.speak()          # Output: Buddy barks.
```

Multiple Inheritance Example:

- In multiple inheritance, a subclass inherits from multiple parent classes.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} makes a sound.") class
Mammal:
    def __init__(self, has_fur):
        self.has_fur = has_fur
    def mammal_info(self):
        print(f"Has fur: {self.has_fur}")
class Dog(Animal, Mammal):
    def __init__(self, name, breed, has_fur):
        Animal.__init__(self, name)  # Initialize Animal class
        Mammal.__init__(self, has_fur) # Initialize Mammal class
        self.breed = breed
        def speak(self):
        print(f"{self.name} barks.")
dog = Dog("Buddy", "Golden Retriever", True)
print(dog.name)         # Output: Buddy
print(dog.breed)        # Output: Golden Retriever
print(dog.has_fur)  # Output: True
dog.speak()          # Output: Buddy barks.
dog.mammal_info()     # Output: Has fur: True
```

Multilevel Inheritance Example:

- In multilevel inheritance, a class is derived from another derived class (i.e., a subclass inherits from another subclass).

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} makes a sound.") class
Mammal(Animal):
    def __init__(self, name, has_fur):
        super().__init__(name)
        self.has_fur = has_fur
    def mammal_info(self):
        print(f"Has fur: {self.has_fur}")
class Dog(Mammal):
    def __init__(self, name, breed, has_fur):
        super().__init__(name, has_fur)
        self.breed = breed
```

```
    def speak(self):
        print(f"{self.name} barks.")
dog = Dog("Buddy", "Golden Retriever", True)
dog.speak()          # Output: Buddy barks.
dog.mammal_info()    # Output: Has fur: True
```

Hierarchical Inheritance Example:

- In hierarchical inheritance, multiple subclasses inherit from the same parent class.

```
class Animal:
    def _init_(self, name):
        self.name = name
    def speak(self):
     print(f"{self.name} makes a sound.") class
Dog(Animal):
    def _init_(self, name, breed):
        super()._init_(name)
        self.breed = breed
    def speak(self):
        print(f"{self.name} barks.")
class Cat(Animal):
    def _init_(self, name, color):
        super()._init_(name)
        self.color = color
    def speak(self):
        print(f"{self.name} meows.")
dog = Dog("Buddy", "Golden Retriever")
cat = Cat("Whiskers", "White") dog.speak()
# Output: Buddy barks. cat.speak()  #
Output: Whiskers meows. Method
```

Resolution Order (MRO):

- When using multiple inheritance, Python uses the Method Resolution Order (MRO) to determine the order in which classes are searched for methods and attributes. This is particularly important when dealing with diamond-shaped inheritance (where a class inherits from two classes, and those classes share a common ancestor).
- You can view the method resolution order using the mro() method:

```
class A:
    def greet(self):
        print("Hello from A")
class B(A):
    def greet(self):
        print("Hello from B")
class C(A):
    def greet(self):
        print("Hello from C")
```

```
class D(B, C):
    pass
print(D.mro())
```

Output:

[<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>]

3.4 Abstract Classes in Python

- An abstract class is a class that cannot be instantiated directly. It serves as a blueprint for other classes. Abstract classes allow you to define methods that must be implemented by the subclasses, but they can also contain methods with default implementations.
- Python provides the abc module (Abstract Base Classes) to define abstract classes and methods.

Syntax:

```
from abc import ABC, abstractmethod
class AbstractClass(ABC):  # Abstract base class inherits from ABC
    @abstractmethod
    def abstract_method(self):
        pass # No implementation here
    def concrete_method(self):
        print("This is a concrete method.")
```

Steps to Create an Abstract Class:

Inherit from ABC: The abstract class must inherit from the ABC (Abstract Base Class) class in the abc module.

Use @abstractmethod decorator: The methods that are meant to be abstract are decorated with the @abstractmethod decorator. These methods must be implemented by any subclass.

Concrete methods: You can define regular methods (concrete methods) with implementation in the abstract class.

Example of abstract class:

```
from abc import ABC, abstractmethod class
Shape(ABC):
    @abstractmethod
    def area(self):
        pass
    @abstractmethod
    def perimeter(self):
        pass
    def display(self):
        print("This is a shape.")
class Rectangle(Shape):
    def _init_(self, length, width):
        self.length = length
```

```
        self.width = width
def area(self):
            return self.length * self.width
        def perimeter(self):
        return 2 * (self.length + self.width) rect
= Rectangle(5, 3)
print("Area of Rectangle:", rect.area()) # Output: 15
print("Perimeter of Rectangle:", rect.perimeter())  # Output: 16
rect.display()                                # Output: This is a shape.
```

Abstract Class with Multiple Subclasses:

Can create multiple subclasses that inherit from the same abstract class and provide different implementations of the abstract methods.

Example:

```
from abc import ABC, abstractmethod class
Shape(ABC):
        @abstractmethod
        def area(self):
            pass
        @abstractmethod
        def perimeter(self):
            pass
class Circle(Shape):
        def _init_(self, radius):
            self.radius = radius
        def area(self):
            return 3.14 * self.radius ** 2
        def perimeter(self):
        return 2 * 3.14 * self.radius
class Square(Shape):
        def _init_(self, side):
            self.side = side
        def area(self):
            return self.side ** 2
        def perimeter(self):
        return 4 * self.side
circle = Circle(4) square
= Square(5)
print("Circle Area:", circle.area())              # Output: 50.24
print("Circle Perimeter:", circle.perimeter()) # Output: 25.12
print("Square Area:", square.area())              # Output: 25
print("Square Perimeter:", square.perimeter()) # Output: 20
```

Why Use Abstract Classes:

Enforce Consistency

Code Reusability

Design Flexibility

3.5 Data Hiding in Python

- Data hiding is a principle of object-oriented programming (OOP) that refers to restricting access to certain attributes or methods of an object to prevent unauthorized or accidental modification.
- In Python, this can be achieved through encapsulation, which is the bundling of data (attributes) and methods (functions) that operate on that data into a single unit, the class.
- While Python does not enforce strict access modifiers like other languages (e.g., private, protected), it uses conventions and name-mangling techniques to achieve data hiding.

Levels of Data Access in Python:

Public Attributes: These attributes are accessible from anywhere.

Protected Attributes: These are meant to be accessed only within the class and its subclasses.

Private Attributes: These are intended to be used only within the class

1. Public Attributes:

- Attributes that are directly accessible and can be modified freely.

```
class Car:
    def _init_(self, make, model):
        self.make = make  # Public attribute
        self.model = model # Public attribute
car = Car("Toyota", "Camry")
print(car.make)       # Output: Toyota
print(car.model)  # Output: Camry
```

2. Protected Attributes:

- In Python, the convention for indicating that an attribute is protected is to prefix it with a single underscore (_).
- This is not a strict mechanism, but it is a convention that suggests that these attributes are intended to be protected and should not be accessed directly outside the class or its subclasses.

```
class Car:
    def __init__(self, make, model):
        self._make = make  # Protected attribute
        self._model = model  # Protected attribute
    def display_info(self):
     print(f"Make: {self._make}, Model: {self._model}")
class ElectricCar(Car):
    def _init_(self, make, model, battery_capacity):
        super()._init_(make, model)
        self_battery_capacity = battery_capacity # Protected attribute
    def display_info(self):
```

```
        super().display_info()
    print(f"Battery Capacity: {self._battery_capacity} kWh")
ev_car = ElectricCar("Tesla", "Model S", 100)
print(ev_car._make) # Output: Tesla (although protected, it is accessible)
print(ev_car._battery_capacity)  # Output: 100
ev_car.display_info()
```

3. Private Attributes:

- To make an attribute private, Python uses name mangling. This means that a variable with a double underscore (__) prefix is intended to be private and not accessed directly outside the class. Python internally changes the name of the attribute to avoid accidental access or modification.

```
class Car:
    def ____init____(self,  make,  model):
        self.__make = make  # Private attribute
        self.__model = model # Private attribute
    def display_info(self):
     print(f"Make: {self.__make}, Model: {self.__model}") car
= Car("Honda", "Civic")
print(car._Car__make) # Output: Honda car.display_info()
# Output: Make: Honda, Model: Civic
```

4. Getters and Setters (Property Methods):

- Getter and setter methods to provide controlled access to private attributes.
- This is a common practice to achieve data hiding.

```
class Car:
    def ____init____(self,  make,  model):
        self.__make = make  # Private attribute
        self.__model = model # Private attribute
    def get_make(self):
        return self.__make
    def set_make(self, make):
        if len(make) > 1: # Example validation
            self.__make = make
        else:
            print("Invalid make.")
    def get_model(self):
        return self.__model
    def set_model(self, model):
        if len(model) > 1: # Example validation
            self.__model = model
        else:
         print("Invalid model.") car
= Car("Toyota", "Camry")
print(car.get_make())       # Output: Toyota
```

```
print(car.get_model())  # Output: Camry
car.set_make("Honda")       # Updates the make to "Honda"
print(car.get_make())       # Output: Honda
car.set_model("Civic")  # Updates the model to "Civic"
print(car.get_model())  # Output: Civic
```

3.6 super() and class Methods in Python

super() in Python

The super() function is used to call methods from a parent class (superclass) from within a subclass. It is typically used to invoke the constructor or methods of the parent class.

Usage of super():

Calling Parent Class Methods: super() allows you to call methods in the parent class, even when they are overridden by the child class.

Accessing Parent Class Constructor: It is commonly used in the child class constructor (__init__) to initialize the parent class.

Syntax of super():

```
super().method_name()
```

Example:

```
class Animal:
    def __init__(self, name):
        self.name = name
        print(f"Animal {self.name} created.")
    def speak(self):
     print(f"{self.name} makes a sound.") class
Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)  # Calling the constructor of the parent class (Animal)
        self.breed = breed
        print(f"Dog breed: {self.breed}")
    def speak(self):
        super().speak() # Calling the parent class method
        print(f"{self.name} barks.")
dog = Dog("Buddy", "Golden Retriever")
dog.speak()
```

Output:

```
Animal Buddy created.
Dog breed: Golden Retriever
Buddy makes a sound.
Buddy barks.
```

Class Method in Python

A class method is a method that is bound to the class rather than an instance of the class. It is defined using the @classmethod decorator. A class method receives the class itself as the first argument, conventionally named cls, instead of the instance (self).

Usage of class Method:

Modifying Class State: A class method can be used to modify the class state or access class-level attributes.

Alternative Constructors: Class methods are often used as alternative constructors, which allow you to create instances in different ways.

Syntax of class Method:

```
class ClassName:
    @classmethod
    def method_name(cls, ...):
        # method body
```

Example:

```
class Animal:
    species = "Mammal" # Class-level attribute
    def _init_(self, name):
        self.name = name
    @classmethod
    def change_species(cls, new_species):
        cls.species = new_species  # Modifies the class-level attribute
    @classmethod
    def create_animal(cls, name):
     return cls(name)  # Alternative constructor using the class method
Animal.change_species("Reptile")
  print(Animal.species)  # Output: Reptile
  animal1 = Animal.create_animal("Lion")
  print(animal1.name)  # Output: Lion
```

Comparison of super() and class Method

- super() is used to access methods from a parent class from within a child class.
- class methods are bound to the class itself and are used to modify class-level attributes or to provide alternative constructors.

```
class Animal:
    species = "Mammal"
    def _init_(self, name):
        self.name = name
    @classmethod
    def change_species(cls, new_species):
        cls.species = new_species
    @classmethod
    def create_animal(cls, name):
        return cls(name)
    def speak(self):
     print(f"{self.name} makes a sound.") class
Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)  # Call the constructor of the parent class
```

```
        self.breed = breed
    @classmethod
    def change_species(cls, new_species):
        super().change_species(new_species) # Call the parent class method
        print(f"Species updated to: {cls.species}")
dog = Dog.create_animal("Buddy")
dog.speak()
# Change the species using class method
dog.change_species("Canine")
# Access class-level attribute 'species'
print(dog.species) # Output: Canine Output:
Buddy makes a sound.
Species updated to: Canine
Canine
```

3.7 Dunder Method

Dunder methods in Python (also known as "magic methods" or "special methods") are methods that have double underscores at the beginning and end of their names, such as _init_, _str_, and _add_. These methods are used to implement behaviour for built-in operations or to customize the behaviour of objects when they interact with Python's syntax and functions.

Here's an overview of some **common dunder methods:**

1. __init__(self, ...)

Purpose: Initializes a new instance of a class. It's the constructor for the object.

2. __str__(self)

Purpose: Defines the string representation of the object. It's used by print() or str().

3. __repr__(self)

Purpose: Similar to __str__, but it is used for "formal" string representations.
It should ideally return a string that can be used to recreate the object.

4. __add__(self, other)

Purpose: Defines behavior for the + operator.

5. __len__(self)

Purpose: Defines behavior for the len() function. It returns the length of an object.

6. __getitem__(self, key)

Purpose: Defines behavior for accessing items using square brackets [].

7. __setitem__(self, key, value)

Purpose: Defines behavior for assigning values to items using square brackets []

8. __del__(self)

Purpose: Defines behavior for when an object is deleted. It is called when an object is about to be destroyed.

9. __eq__(self, other)

Purpose: Defines behavior for equality comparison (==).

10. __iter__(self)

Purpose: Defines behavior for iteration, such as when using for loops.

3.8 Static and Instance Method

1. Instance Methods

Definition: Instance methods are the most common type of method in Python. These methods operate on an instance of the class (an object), and they have access to the instance's attributes and other methods.

Binding: Instance methods are bound to an instance of the class. They take the instance (self) as their first argument.

Use: Instance methods are typically used to access and modify the attributes of the object and to define behaviour specific to an instance.

Example:

```
class Dog:
    def _init_(self, name, age):
        self.name = name
        self.age = age
    def bark(self):
        print(f"{self.name} says Woof!")
    def get_age(self):
     return self.age  dog
= Dog("Buddy", 3)
dog.bark() # Output: Buddy says Woof!
print(dog.get_age())  # Output: 3
```

2. Static Methods

Definition: Static methods don't operate on an instance of the class or modify the class state. They are bound to the class itself and can be called on the class without creating an instance. Static methods don't have access to self or cls (the class reference).

Binding: Static methods are not bound to the instance, and they do not take self as their first argument.They are defined using the @staticmethod decorator.

Use: Static methods are often used for utility functions that are related to the class but don't require access to the class or instance data.

Example:

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b
    @staticmethod
    def multiply(a, b):
     return a * b print(Math.add(5,
3))  # Output: 8
print(Math.multiply(5, 3))  # Output: 15
```

Key Differences Between Instance Methods and Static Methods

Feature	Instance Method	Static Method
Binding	Bound to an instance (self)	Not bound to any instance or class
First Parameter	Takes self as the first parameter	No self parameter
Access to Instance Data	Can access instance attributes and methods	Cannot access instance attributes
Call Method	Called via instance (obj.method())	Called via class (Class.method())
Use Case	Used when you need to access or modify instance data	Used when method logic doesn't depend on instance data

3.9 Self Parameter

In Python, the self parameter refers to the instance of the class that is calling a method. It is used in instance methods to access attributes and other methods of the current object. The self parameter is a convention, not a keyword in Python, but it is highly recommended and almost universally used in Python programming.

What is self?

self is the first parameter of instance methods in a class.
It refers to the instance of the class that the method is being called on.
It allows the method to access instance attributes and other methods within the class.

How Does self Work?

When you define a method inside a class, you must include self as the first parameter. When you call this method on an instance, Python automatically passes the instance to self.

Example : self in Instance Methods

```
class Dog:
    def __init__(self, name, age):
        self.name = name # instance variable
        self.age = age    # instance variable
    def bark(self):
        print(f"{self.name} says Woof!")
    def get_age(self):
     return self.age  dog
= Dog("Buddy", 3)
dog.bark()            # Output: Buddy says Woof!
print(dog.get_age()) # Output:
```

Why Do We Use self?

Distinguishes Instance Attributes:

It helps distinguish between instance attributes and local variables or parameters.

For instance, the self.name refers to an attribute of the instance, while name might refer to a local variable.

Accessing Object's Data:

The self parameter allows instance methods to access and modify the object's attributes.

This makes the methods operate on the object they belong to, ensuring that each object has its own independent state.

Required by Convention:

Python's object-oriented structure requires that instance methods include self as the first parameter, though it is not a keyword—just a convention.

What Happens Behind the Scenes?

Method Call:

When you call a method on an instance, like dog.bark(), Python internally translates this into Dog.bark(dog), passing the instance (dog) to the method.

Automatic self Passing:

You don't need to manually pass the self parameter; Python handles this automatically. You only pass the arguments to the method after self.

Example:

```
class Person:
    def __init__(self, name):
        self.name = name
    def greet(self):
     print(f"Hello, {self.name}!")
person = Person("Alice") person.greet()
# Output: Hello, Alice!
```

self and Class Methods:

Class methods and static methods do not require self.

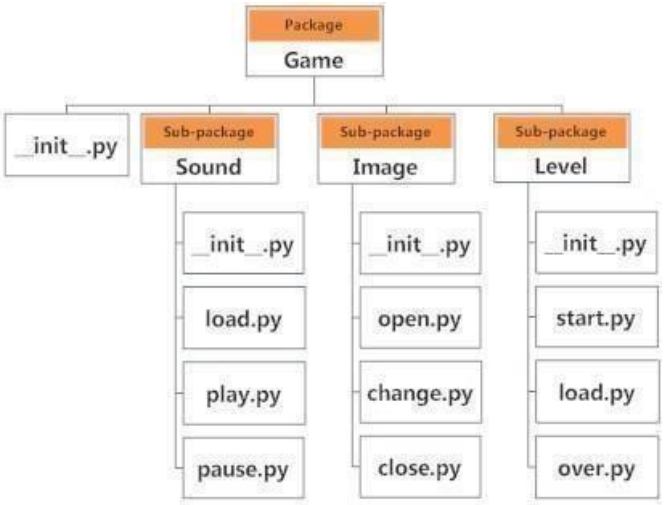
Instead, class methods use cls (the class itself), and static methods do not require any reference to the class or instance at all.

Example of class method (cls):

```
class Dog:
    num_of_dogs = 0
    def __init__(self, name):
        self.name = name
        Dog.num_of_dogs += 1
    @classmethod
    def total_dogs(cls):
        return cls.num_of_dogs
print(Dog.total_dogs())  # Output: 0 (before creating any instance) d1
= Dog("Buddy")
d2 = Dog("Max")
print(Dog.total_dogs()) # Output: 2
```

3.10 Developing and Deploying Packages

- A package is a collection of modules. A Python package can have sub-packages and modules.
- A directory must contain a file named init.py in order for Python to consider it as a
- package. This file can be left empty but we generally place the initialization code for that package in this file.
- Here is an example. Suppose we are developing a game, one possible organization of
- packages and modules could be as shown in the figure below.



Importing module from a package

- We can import modules from packages using the dot (.) operator.
- For example, if want to import the start module in the above example, it is done as follows

  import Game.Level.start
- Now if this module contains a function named select_difficulty(), we must use the full Name to reference it.

  Game.Level.start.select_difficulty(2)

If this construct seems lengthy, we can import the module without the package prefix as follows.

from Game.Level import start

We can now call the function simply as follows.

start.select_difficulty(2)

Yet another way of importing just the required function (or class or variable) form a module within a package would be as follows.

from Game.Level.start import select_difficulty Now we can directly call this function. *select_difficulty(2)*

Although easier, this method is not recommended. Using the full namespace avoids confusion and prevents two same identifier names from colliding.

While importing packages, Python looks in the list of directories defined in sys.path, similar as for module search path.

UNIT 4

PYTHON FILE HANDLING

File Objects and Methods - Opening and Closing of a File - Input and Output Operations – Handling of Binary Files – Sequential and Random Access Files – Reading CSV Files - Writing CSV Files.

4.1 File Objects and Methods

A file is a collection of data stored on a disk in **text** or **binary** format. Python provides built-in functions to perform operations on files, such as **reading, writing, and appending data**.

4.1.1 File Objects in Python

A **file object** provides methods and attributes to work with files in Python. Files can be **opened, read, written, and closed** using built-in functions.

Syntax of open() function:

file_object = open("filename", "mode")

- filename: Name of the file (with extension) to be opened.
- mode: Specifies the mode in which the file is opened.

File Opening Modes:

Mode	Description
"r"	Read mode (default). Opens file for reading.
"w"	Write mode. Creates a new file if it doesn't exist, otherwise overwrites it.
"a"	Append mode. Adds data to the end of the file.
"r+"	Read and write mode. Reads existing content and allows modifications.
"w+"	Write and read mode. Overwrites existing file or creates a new one.
"a+"	Append and read mode. Reads the file and appends new content.
"rb", "wb"	Read/Write binary mode. Used for images, videos, etc.

4.2 Opening a File in Python

The open()function in Python is used to open a file for reading, writing, or appending. It returns a **file object**, which allows interaction with the file (reading/writing).

Syntax:

file_object = open("filename", "mode")

- "filename": The name of the file to open.

- "mode": Specifies how the file should be opened.

Example:

```
# Opening a file in read mode

file = open("example.txt",

"r") print("File opened

successfully") # Closing the

file

file.close()
```

Output:

File opened successfully

Explanation:

1. `open("example.txt", "r")`opens the file in **read mode**.
2. `close()`is called to release system resources after reading.

Closing a File in Python

After performing file operations, it is important to close the file using close(), which releases system resources.

Example:

```
file = open("example.txt",

"r") print("File is opened")

file.close()  # Closing the

file print("File is closed")
```

Output:

File is opened

File is closed

4.3 Input and Output Operations

Python provides built-in functions for **file input and output (I/O) operations**, allowing us to **read from and write to files** easily.

4.3.1 Writing to a File (write() method)

To write data to a file, we use the write() or write lines() methods.

Example: Writing to a File

```
# Open file in write mode
file = open("example.txt", "w")

# Writing to the file
file.write("Hello,
Python!\n")
file.write("File handling is easy in Python.\n")

# Close the file
file.close()

print("Data written successfully.")
```

Output:

Data written successfully.

File Content (example.txt)

Hello, Python!

File handling is easy in Python.

Explanation:

- The file "example.txt" is opened in "w" (write) mode.
- The write() method writes text into the file.
- The close() method ensures that the file is saved properly.

4.3.2 Reading from a File

Python provides multiple methods to read a file:

- **read()**
- **readline()**
- **readlines())**

Methods	Description
read(size)	Reads the entire file or a specific number of bytes
readline()	Reads one line at a time
readlines()	Reads all lines into a list.

Example 1: Reading an Entire File (read())

```
file = open("example.txt", "r")

content = file.read() # Reads the entire

file print("File Content:\n", content)

file.close()
```

Output:

```
File Content:

 Hello, Python!

 File handling is easy in Python.
```

Explanation:

- The file is opened in "r" (read) mode.
- The read() method reads the entire file content.
- The file is then closed.

Example 2: Reading Line by Line (readline())

```
file = open("example.txt", "r")

print(file.readline())  # Reads the first line

print(file.readline())  # Reads the second line

file.close()
```

Output:

```
Hello, Python!

File handling is easy in Python.
```

Explanation:

- The readline() method reads one line at a time.
- Each call to readline() moves the cursor to the next line.

Example 3: Reading All Lines as a List (readlines())

```
file = open("example.txt", "r")

lines = file.readlines()  # Reads all lines into a list
```

```
print(lines)   file.close()
```

Output:

```
['Hello, Python!\n', 'File handling is easy in Python.\n']
```

Explanation:

- The readlines() method reads all lines and stores them in a list.

4.3.3 Appending to a File (a mode)

The append mode ("a") allows adding new content to an existing file without overwriting it.

```
file = open("example.txt", "a")  # Open in append mode
file.write("Appending new line!\n")
file.close()
# Reading the file after appending
file = open("example.txt", "r")
print(file.read())
file.close()
```

Output:

```
Hello, Python!
File handling is easy in Python.
Appending new line!
```

Explanation:

- The "a" mode ensures that the new line is added to the file without deleting existing content.

4.3.4 Using with open() Statement

Using with open(), files are automatically closed after use.

Example: Using with open()

```
with open("example.txt", "r") as file:
    content = file.read()
    print(content)
```

Output:

Hello, Python!

File handling is easy in Python.

Appending new line!

Explanation:

- The with open() statement ensures that the file is **automatically closed** after execution.

4.3.5 Writing Multiple Lines to a File (writelines())

The writelines() method writes multiple lines at once.

Example: Writing Multiple Lines

```
lines = ["Python is fun!\n", "Learning file handling.\n", "Appending new data.\n"] with

open("example.txt", "w") as file:

    file.writelines(lines)

# Reading the modified file

with open("example.txt", "r") as file: print(file.read())
```

Output:

Python is fun!

Learning file handling.

Appending new data.

4.4 Handling of Binary Files: Sequential and Random Access Files

Binary files store data in binary format, making them more efficient for storing and retrieving structured data (like images, audio, and serialized objects).

4.4.1 Sequential Access in Binary Files

- In **sequential access**, data is written and read in a linear order.
- Reading and writing operations start from the beginning and progress forward.
- Used in applications where data is processed in a predetermined sequence.

Example: Writing and Reading a Binary File Sequentially

```
# Writing to a binary file sequentially

with open("data.bin", "wb") as file:

    numbers = [10, 20, 30, 40, 50]

    byte_data = bytearray(numbers)  # Convert list to byte

    array file.write(byte_data)

# Reading from a binary file

sequentially with open("data.bin",

"rb") as file:

    data = file.read()  # Read the entire file

    numbers_list = list(data)  # Convert bytes back to list

    print("Read Data:", numbers_list)
```

Output:

```
Read Data: [10, 20, 30, 40, 50]
```

Explanation:

1. The program writes a list of numbers into a binary file using write().
2. The program then reads the entire file back using read().
3. The retrieved data is converted from bytes to a list before printing.

4.4.2 Random Access in Binary Files

- In **random access**, data can be read or written at any position in the file without following a sequence.
- This is useful for modifying specific parts of a large file without reading the whole file into memory.

Example: Random Access File Handling

```
import struct

# Writing to a binary file with random

access with open("random.bin", "wb") as

file:

    data = [100, 200, 300, 400, 500]

    for num in data:

        file.write(struct.pack("i", num)) # Writing integers in binary

format # Reading a specific record using random access

with open("random.bin", "rb") as file:

    file.seek(2 * struct.calcsize("i")) # Move to the 3rd integer (zero-based

    index) value = struct.unpack("i", file.read(struct.calcsize("i")))[0]

    print("Value at 3rd position:", value)
```

Output:

```
Value at 3rd position: 300
```

Explanation:

1. The program writes five integers to a binary file using struct.pack(), which converts integers into binary format.
2. The seek() function moves the file pointer to the required position.
3. The unpack() function reads and converts binary data back to an integer.

Differences between sequential and random access:

Feature	Sequential Access	Random Access
Access Type	Linear	Direct (jump to any location)
Speed	Slower for large files	Faster for large files
Best for	Streaming data (logs, media)	Modifying structured data (databases, indices)

Examples:

- **Sequential Access**: Processing logs, reading images.
- **Random Access**: Updating records in databases, modifying large binary files.

4.5 Reading and Writing CSV Files in Python

A **CSV (Comma-Separated Values) file** is a simple text file used to store tabular data. Python provides built-in support for handling CSV files using the csv module.

4.5.1 Reading CSV Files

Python provides multiple ways to read CSV files usding the csv.reader() and DictReader() functions.

Example: Reading a CSV File Using csv.reader()

```
import csv

# Open and read a CSV file
with open("sample.csv", "r") as file:
   reader = csv.reader(file) # Create a reader
   object for row in reader:
      print(row)  # Each row is a list
```

Example CSV File (sample.csv):

```
Name,Age,Departme
nt Alice,25,HR
Bob,30,IT
Charlie,28,Finance
```

Output:

```
['Name', 'Age', 'Department']
['Alice', '25', 'HR']
['Bob', '30', 'IT']
['Charlie', '28', 'Finance']
```

Explanation:

1. The csv.reader() function reads the CSV file row by row.
2. Each row is returned as a list of values.
3. The first row typically contains column headers.

Example: Reading a CSV File as a Dictionary Using csv.DictReader()

```
import csv

# Open and read CSV file using DictReader
with open("sample.csv", "r") as file:
   reader = csv.DictReader(file) # Read as
   dictionary for row in reader:
      print(row)  # Each row is an OrderedDict
```

Output:

```
{'Name': 'Alice', 'Age': '25', 'Department': 'HR'}
{'Name': 'Bob', 'Age': '30', 'Department': 'IT'}
{'Name': 'Charlie', 'Age': '28', 'Department': 'Finance'}
```

Explanation:

1. csv.DictReader() treats the first row as column headers.
2. It returns each row as an **OrderedDict**, where column headers are keys.

4.5.2 Writing to CSV Files

Python provides the csv.writer() and DictWriter() methods for writing data to CSV files

Example: Writing a CSV File Using csv.writer()

```
import csv

# Data to write
data = [
    ["Name", "Age", "Department"],
    ["David", 32, "Marketing"],
    ["Eve", 27, "Sales"],
    ["Frank", 29, "Engineering"]
]

# Writing data to CSV file
with open("output.csv", "w", newline="") as file:
    writer = csv.writer(file) # Create a writer object
    writer.writerows(data)  # Write multiple rows
print("CSV file written successfully.")
```

Output File (output.csv):

```
Name,Age,Department
David,32,Marketing
Eve,27,Sales
Frank,29,Engineering
```

Explanation:

1. The csv.writer() object is used to write data to the CSV file.
2. writerows() writes multiple rows at once.

Example: Writing a CSV File Using csv.DictWriter()

```
import csv

# Data
to write
data = [
    {"Name": "George", "Age": 35, "Department": "HR"},
    {"Name": "Hannah", "Age": 29, "Department": "IT"},
    {"Name": "Isaac", "Age": 40, "Department": "Finance"}
]

# Writing data as a dictionary
with open("output_dict.csv", "w",
    newline="") as file: fieldnames =
    ["Name", "Age", "Department"]
    writer = csv.DictWriter(file, fieldnames=fieldnames)

    writer.writeheader()  # Write
    header row writer.writerows(data)
    # Write data rows

print("CSV file written successfully.")
```

Output File (output_dict.csv):

```
Name,Age,Depar
tment
George,35,HR
Hannah,29,IT
Isaac,40,Finance
```

Explanation:

1. csv.DictWriter() writes dictionaries instead of lists.
2. writeheader() writes the header row from dictionary keys.
3. writerows() writes each dictionary as a row.

Differences Reading and Writing CSV Files

Method	Reading	Writing
csv.reader()	Returns each row as a list	Uses csv.writer() to write lists
csv.DictReader()	Returns each row as a dictionary	Uses csv.DictWriter() to write dictionaries

UNIT –V PYTHON WEB APPLICATION FRAMEWORK

Basics of MySQL – Database Creation – Tables – MySQL Commands – SELECT – Where Clause – DELETE – UPDATE - Connect Python to MySQL - Flask Introduction – Flask Setup – Flask App Routing – URL Building – HTTP Methods – Templates – Request Body – Developing Web Applications connecting My SQL.

5.1 Basics of MYSQL

What is a Database?

A **database** is a digital system designed for the **storage** and **arrangement** of data. Think of it as an online filing system that allows you to store and quickly access a vast amount of information. Databases facilitate the efficient management of data, enabling the simple **addition**, **modification**, **removal**, and **access** of information. They serve numerous uses such as websites, applications, and enterprises to manage extensive data in an organized and secure manner.

What is MySQL?

MySQL is an open-source relational database management system **(RDBMS)** that uses Structured Query Language (SQL) to manage data. Developed by MySQL AB and now owned by Oracle Corporation, it's widely used due to its **reliability**, **speed**, and **ease** of use. MySQL is a key component in many web applications, forming the backbone of popular websites and services.

- It allows users to create, modify, and maintain databases, supporting operations like data insertion, querying, updating, and deletion. Ideal for both small and large-scale applications, MySQL powers various types of systems, from personal projects to complex enterprise environments.
- MySQL is a Relational Database Management System (RDBMS) software that provides many features, which are as follows:
- **Data Storage**: Efficiently stores large amounts of data.
- **Data Retrieval**: Allows quick and easy access to data.
- **Data Manipulation**: Supports operations like inserting, updating, and deleting data.
- **Data Security**: Offers robust security features to protect data.
- **Scalability**: Can handle small to large applications with ease

How MySQL Works?

- MySQL works like this:

1. **Client Request**: You send a request to the MySQL server using an application or a command-line tool.
2. **Connection**: The server connects with your application to start a session.
3. **SQL Parsing**: The server checks your SQL query for any errors.
4. **Query Optimization**: The server figures out the best way to execute your query efficiently.
5. **Execution**: The server runs the query. It reads or writes data to the database as needed.
6. **Storage Engine**: Data is stored on the disk. MySQL uses different storage engines like InnoDB or MyISAM for this.
7. **Result Generation**: The server creates the result based on your query.
8. **Response**: The server sends the results back to your application.
9. **Client Interaction**: Your application displays the data to you.
10. **Transaction Management**: For complex operations, MySQL ensures everything runs smoothly.

5.2 DATABASE CREATION

SQL CREATE DATABASE:

The CREATE DATABASE statement is a foundational SQL command used to **create new databases** in SQL-based Database Management Systems (DBMS), including **MySQL**, **SQL Server**, and others.

CREATE DATABASE in SQL

The CREATE DATABASE command establishes a new **database** within your **SQL** ecosystem. A database is a repository that organizes data in structured formats through **tables**, views, **stored procedures**, and other components.

Syntax:

The syntax to use the CREATE DATABASE command in SQL is:

CREATE DATABASE database_name;

SQL CREATE DATABASE Example:

How to create a database in SQL and **how to verify database creation** by listing databases in the server.

Creating Your First SQL Database

To create a new database in SQL we use the **CREATE DATABASE** command and then we mention the name of the database. Note that blank spaces are not allowed in the name of the database, we can only use underscore (_).

As an example, we will create a new database with the name "GeeksForGeeks".

SQL Query:

CREATE DATABASE GeeksForGeeks;

Output

```
mysql> CREATE DATABASE GEEKSFORGEEKS;
Query OK, 1 row affected (0.02 sec)

mysql> |
```

Creating a new database

Database Successfully Created!!

List Databases in SQL:

The new database that we have just created has been successfully added to our system or not.

We use the **SHOW DATABASES** command and it will return a list of databases that exist in our system. Now we will look for the name of the database (**GeeksForGeeks**) that we have just created.

SQL Query:

SHOW DATABASES;

Output

```
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| book_shop          |
| employee_directory |
| geeksforgeeks      |
| ig_clone           |
| information_schema |
| inst_clone         |
| mysql              |
| performance_schema |
| sakila             |
```

Database successfully created

As we can see the SHOW DATABASE command returned the list of all databases that exist in our system and in that list, the name of the database (GeeksForGeeks) that we have just created has also been added which verifies that the database has been successfully created.

USE Database in SQL

Once your database is created, we can switch to that database to begin adding tables, inserting data, and performing queries. To do this, use the USE command.

Syntax:

The syntax to use a database in SQL is:

```
USE database_name
```

After running this command, any subsequent queries will apply to the GeeksForGeeks database. This allows you to work within the scope of the newly created database.

Delete a Database in SQL

If you ever need to remove a database, the DROP DATABASE command can be used to delete the database and all its contents:

```
DROP DATABASE GeeksForGeeks;
```

Be cautious when using this command, as it permanently deletes the database and all of its data.

Important Points About SQL CREATE DATABASE

- **Database Naming Conventions:** When establishing a database, it is essential to ensure that the name is
- unique within your SQL environment. While most SQL implementations treat database names as case-insensitive, maintaining consistency in the naming format is advisable. Avoid using spaces; instead, opt for underscores (_). Example: my_database is preferable to my database.
- **Maximum Length of Database Names:** The typical maximum length for a database name in SQL is 128 characters. Ensure that your database name is both clear and concise while complying with this limit.
- **Permissions:** It is imperative to possess the requisite privileges to create databases within your SQL environment. In many systems, the CREATE DATABASE command can only be executed by users with administrative privileges.
- **Case Insensitivity:** Most SQL DBMSs consider database names to be case-insensitive; however, it is recommended to consistently use lowercase or follow a uniform formatting style throughout your environment.
- **Compatibility Across SQL Databases:** While the CREATE DATABASE statement is generally similar across various SQL databases such as MySQL and SQL Server, be mindful of minor syntactical differences that may exist in specific systems.

5.3 TABLES

HTML Tables:

An HTML table is created using the <table> tag. Inside this tag, you use

- ***<tr>*** *to define table rows,*
- ***<th>*** *for table headers, and*
- ***<td>*** *for table data cells*

Each <tr> represents a row, and within each row, <th> or <td> tags represent the cells in that row, which can contain text, images, lists, or even another table.

HTML Table Code Example:

```
<!-- index.html -->
<!DOCTYPE html>
<html>

<body>
<table>
<tr>
<th>Firstname</th>
<th>Lastname</th>
<th>Age</th>
</tr>
<tr>
<td>Priya</td>
<td>Sharma</td>
<td>24</td>
</tr>
<tr>
<td>Arun</td>
<td>Singh</td>
<td>32</td>
</tr>
<tr>
<td>Sam</td>
<td>Watson</td>
<td>41</td>
</tr>
</table>
</body>

</html>
```

Output:

Firstname	Lastname	Age
Priya	Sharma	24
Arun	Singh	32
Sam	Watson	41

Simple HTML Table

In this Example:

- <table>: This tag starts the table. Everything between the opening <table> and closing </table> tags makes up the table.
- <tr>: Stands for "table row". Each <tr> tag defines a row in the table.
- <th>: Stands for "table header". It's used for the headers of the columns. In this case, "**Firstname**", "**Lastname**", and "**Age**" are headers. Text in <th> tags is usually bold and centered by default.
- <td>: Stands for "table data". This tag is used for actual data cells under each column. For instance, "**Priya**" is the data under the "**Firstname**" header, "**Sharma**" under the "**Lastname**", and "**24**" under the "**Age**".
- The first <tr> has three <th> elements, setting up the column titles.
- The subsequent <tr> tags each contain three <td> elements, representing the data for each person listed in the table.

  When this HTML code is rendered in a web browser, it will display a table with four rows (one header row plus three data rows) and three columns (Firstname, Lastname, Age), showing the names and ages of Priya, Arun, and Sam.

Tags used in HTML Tables:

HTML Tags	Descriptions
<table>	Defines the structure for organizing data in rows and columns within a web page.
<tr>	Represents a **row** within an HTML table, containing individual cells.
<th>	Shows a table **header** cell that typically holds titles or headings.
<td>	Represents a standard **data** cell, holding data.
<caption>	Provides a title or description for the entire table.

<thead>	Defines the header section of a table, often containing column labels.
<tbody>	Represents the main content area of a table, separating it from the header or footer.
<tfoot>	Specifies the footer section of a table, typically holding summaries or totals.
<col>	Defines attributes for table **columns** that can be applied to multiple columns at once.
<colgroup>	Groups together a **set of columns** in a table to which you can apply formatting or properties collectively.

Another Example of HTML Table:
Creating a simple table in HTML using a table tag.

```
<!-- index.html -->
<!DOCTYPE html>
<html>

<body>
<table>
<tr>
<th>Book Name</th>
<th>Author Name</th>
<th>Genre</th>
</tr>
<tr>
<td>The Book Thief</td>
<td>Markus Zusak</td>
<td>Historical Fiction</td>
</tr>
<tr>
<td>The Cruel Prince</td>
<td>Holly Black</td>
<td>Fantasy</td>
</tr>
<tr>
<td>The Silent Patient</td>
<td> Alex Michaelides</td>
```

```
<td>Psychological Fiction</td>
</tr>
</table>
</body>
</html>
```

Output:

Book Name	Author Name	Genre
The Book Thief	Markus Zusak	Historical Fiction
The Cruel Prince	Holly Black	Fantasy
The Silent Patient	Alex Michaelides	Psychological Fiction

HTML Table

Styling HTML Tables

Styling an HTML table can significantly improve its appearance and readability. You can use CSS (Cascading Style Sheets) to add styles such as borders, background colors, text alignments, and much more.

1. Adding a Border to an HTML Table

A border is set using the CSS border property. If you do not specify a border for the table, it will be displayed without borders.

Syntax

```
table, th, td {
    border: 1px solid black;
}
```

Example: Addition of the border to the HTML Table.

```
<html>
<head>
<style>
table,
th,
td {
border: 1px solid black;
}
</style>
</head>

<body>
<table style="width:100%">
<tr>
<th>Firstname</th>
<th>Lastname</th>
<th>Age</th>
</tr>
<tr>
<td>Priya</td>
```

```
<td>Sharma</td>
<td>24</td>
</tr>
<tr>
<td>Arun</td>
<td>Singh</td>
<td>32</td>
</tr>
<tr>
<td>Sam</td>
<td>Watson</td>
<td>41</td>
</tr>
</table>
</body>
</html>
```

Output:

Firstname	Lastname	Age
Priya	Sharma	24
Arun	Singh	32
Sam	Watson	41

Fig:HTML Table with border

2. Adding Collapsed Borders in an HTML Table

For borders to collapse into one border, add the CSS border-collapse property.

Syntax

```
table, th, td {

    border: 1px solid black;

    border-collapse: collapse;

  }
```

Example: Addition of Collapsed Borders in HTML.

```
<html>
<head>
<style>
table,
th,
td {
border: 1px solid black;
border-collapse: collapse;
}
</style>
</head>
```

```
<body>
<table style="width:100%">
<tr>
<th>Firstname</th>
<th>Lastname</th>
<th>Age</th>
</tr>
<tr>
<td>Priya</td>
<td>Sharma</td>
<td>24</td>
</tr>
<tr>
<td>Arun</td>
<td>Singh</td>
<td>32</td>
</tr>
<tr>
<td>Sam</td>
<td>Watson</td>
<td>41</td>
</tr>
</table>
</body>

</html>
```

Output:

Firstname	Lastname	Age
Priya	Sharma	24
Arun	Singh	32
Sam	Watson	41

Fig:HTML Table with Collapsed Borders

3. Adding Cell Padding in an HTML Table

Cell padding specifies the space between the cell content and its borders. If we do not specify a padding, the table cells will be displayed without padding.

Syntax

```
th, td {

    padding: 20px;

}
```

Example: Addition of Table cell padding in HTML.

```
<html>
<head>
```

```
<style>
table,
th,
td {
border: 1px solid black;
border-collapse: collapse;
}

th,
td {
padding: 20px;
}
</style>
</head>
<body>
<table style="width:100%">
<tr>
<th>Firstname</th>
<th>Lastname</th>
<th>Age</th>
</tr>
<tr>
<td>Priya</td>
<td>Sharma</td>
<td>24</td>
</tr>
<tr>
<td>Arun</td>
<td>Singh</td>
<td>32</td>
</tr>
<tr>
<td>Sam</td>
<td>Watson</td>
<td>41</td>
</tr>
</table>
</body>

</html>
```

Output:

Firstname	Lastname	Age
Priya	Sharma	24
Arun	Singh	32
Sam	Watson	41

Fig:Adding Table cell padding

4. Adding Left Align Headings in an HTML Table

The table headings are bold and centered. To left-align the table headings, we must use the CSS text-align property.

Syntax

```
th {

    text-align: left;

}
```

Example: Explains the text-align property where the text is aligned to the left.

```
<html>
<head>
<style>
table,
th,
td {
border: 1px solid black;
border-collapse: collapse;
}

th,
td {
padding: 20px;
}

th {
text-align: left;
}
</style>
</head>

<body>
```

```
<table style="width:100%">
<tr>
<th>Firstname</th>
<th>Lastname</th>
<th>Age</th>
</tr>
<tr>
<td>Priya</td>
<td>Sharma</td>
<td>24</td>
</tr>
<tr>
<td>Arun</td>
<td>Singh</td>
<td>32</td>
</tr>
<tr>
<td>Sam</td>
<td>Watson</td>
<td>41</td>
</tr>
</table>
</body>
</html>
```

Output:

Firstname	**Lastname**	**Age**
Priya	Sharma	24
Arun	Singh	32
Sam	Watson	41

Fig: text-align Property

5. Adding Border Spacing in an HTML Table

Border spacing specifies the space between the cells. To set the border-spacing for a table, we must use the CSS border-spacing property.

Syntax

```
table {
    border-spacing: 5px;
}
```

Example: Explains the <u>border space</u> property to make the space between the Table cells.

```
<html>
<head>
```

```
<style>
table,
th,
td {
border: 1px solid black;
}

table {
border-spacing: 5px;
}
</style>
</head>

<body>
<table style="width:100%">
<tr>
<th>Firstname</th>
<th>Lastname</th>
<th>Age</th>
</tr>
<tr>
<td>Priya</td>
<td>Sharma</td>
<td>24</td>
</tr>
<tr>
<td>Arun</td>
<td>Singh</td>
<td>32</td>
</tr>
<tr>
<td>Sam</td>
<td>Watson</td>
<td>41</td>
</tr>
</table>
</body>

</html>
```

Output:

Firstname	Lastname	Age
Priya	Sharma	24
Arun	Singh	32
Sam	Watson	41

Fig:Border Spacing Property

6. Adding Cells that Span Many Columns in HTML Tables

To make a cell span more than one column, we must use the colsp an attribute.

Example: Use of colsp an attribute in HTML.

```
<html>
<head>
<style>
table,
th,
td {
border: 1px solid black;
border-collapse: collapse;
}

th,
td {
padding: 5px;
text-align: left;
}
</style>
</head>

<body>
<h2>Cell that spans two columns:</h2>
<table style="width:100%">
<tr>
<th>Name</th>
<th colspan="2">Telephone</th>
</tr>
<tr>
<td>Vikas Rawat</td>
<td>9125577854</td>
<td>8565557785</td>
</tr>
</table>
</body>

</html>
```

Output:

Cell that spans two columns:

Name	Telephone	
Vikas Rawat	9125577854	8565557785

Fig:colspan attribute

7. Adding Cells that span many rows in HTML Tables

To make a cell span more than one row, we must use the rowspan attribute.

Example: Use of the rowspan attribute in HTML.

```
<html>
<head>
<style>
table,
th,
td {
border: 1px solid black;
border-collapse: collapse;
}

th,
td {
padding: 5px;
text-align: left;
}
</style>
</head>

<body>
<h2>Cell that spans two rows:</h2>
<table style="width:100%">
<tr>
<th>Name:</th>
<td>Vikas Rawat</td>
</tr>
<tr>
<th rowspan="2">Telephone:</th>
<td>9125577854</td>
</tr>
<tr>
<td>8565557785</td>
</tr>
</table>
</body>
```

```
</html>
```

Output:

Cell that spans two rows:

Name:	Vikas Rawat
Telephone:	9125577854
	8565557785

Fig:Use of rowspan attribute

8. Adding a Caption in an HTML Table

To add a caption to a table, we must use the "caption" tag.

Syntax

```
<table style="width:100%">
<caption>DETAILS</caption>
```

Example: HTML Table caption by specifying the CSS properties for setting its width.

```
<html>
<head>
<style>
table,
th,
td {
border: 1px solid black;
border-collapse: collapse;
}
th,
td {
padding: 20px;
}

th {
text-align: left;
}
</style>
</head>

<body>
<table style="width:100%">
<caption>DETAILS</caption>
<tr>
<th>Firstname</th>
<th>Lastname</th>
<th>Age</th>
</tr>
<tr>
```

```
<td>Priya</td>
<td>Sharma</td>
<td>24</td>
</tr>
<tr>
<td>Arun</td>
<td>Singh</td>
<td>32</td>
</tr>
<tr>
<td>Sam</td>
<td>Watson</td>
<td>41</td>
</tr>
</table>
</body>

</html>
```

Output:

DETAILS

Firstname	**Lastname**	**Age**
Priya	Sharma	24
Arun	Singh	32
Sam	Watson	41

Fig: Adding the caption using the tag

9. Adding a Background Colour to the Table

A color can be added as a background in an HTML table using the "*background-color*" option.

Syntax

```
table#t01 {

        width: 100%;

        background-color: #f2f2d1;

 }
```

Example: Addition of the Table background color in HTML.

```
<!-- index.html -->
<!DOCTYPE html>
<html>
```

```
<head>
<style>
table,
th,
td {
border: 1px solid black;
border-collapse: collapse;
}

th,
td {
padding: 5px;
text-align: left;
}

table#t01 {
width: 100%;
background-color: #f2f2d1;
}
</style>
</head>

<body>
<table style="width:100%">
<tr>
<th>Firstname</th>
<th>Lastname</th>
<th>Age</th>
</tr>
<tr>
<td>Priya</td>
<td>Sharma</td>
<td>24</td>
</tr>
<tr>
<td>Arun</td>
<td>Singh</td>
<td>32</td>
</tr>
<tr>
<td>Sam</td>
<td>Watson</td>
<td>41</td>
</tr>
</table>
<br />
```

```
<br />
<table id="t01">
<tr>
<th>Firstname</th>
<th>Lastname</th>
<th>Age</th>
</tr>
<tr>
<td>Priya</td>
<td>Sharma</td>
<td>24</td>
</tr>
<tr>
<td>Arun</td>
<td>Singh</td>
<td>32</td>
</tr>
<tr>
<td>Sam</td>
<td>Watson</td>
<td>41</td>
</tr>
</table>
</body>

</html>
```

Output:

Firstname	Lastname	Age
Priya	Sharma	24
Arun	Singh	32
Sam	Watson	41

Firstname	Lastname	Age
Priya	Sharma	24
Arun	Singh	32
Sam	Watson	41

Fig:Adding Table Background color using CSS properties

10. Creating Nested Tables

Nesting tables simply means making a Table inside another Table

Example: Nesting of HTML Table.

```
<html>
<body>
```

```
<table border=5 bordercolor=black>
<tr>
<td> First Column of Outer Table </td>
<td>
<table border=5 bordercolor=grey>
<tr>
<td> First row of Inner Table </td>
</tr>
<tr>
<td> Second row of Inner Table </td>
</tr>
</table>
</td>
</tr>
</table>
</body>
</html>
```

Output:

Fisrt Column of Outer Table	First row of Inner Table Second row of Inner Table

Fig: Nested HTML Table

5.4 MYSQL Commands

Comments are used to explain sections of SQL statements, or to prevent execution of SQL statements.

Single Line Comments:

Single line comments start with --.

Any text between -- and the end of the line will be ignored (will not be executed).

The following example uses a single-line comment as an explanation:

Example

```
-- Select all:
SELECT * FROM Customers;
```

The following example uses a single-line comment to ignore the end of a line:

Example

```
SELECT * FROM Customers -- WHERE City='Berlin';
```

The following example uses a single-line comment to ignore a statement:

Example

```
-- SELECT * FROM Customers;
SELECT * FROM Products;
```

Multi-line Comments

Multi-line comments start with /* and end with */.

Any text between /* and */ will be ignored.

The following example uses a multi-line comment as an explanation:

Example

```
/*Select all the columns
of all the records
in the Customers table:*/
SELECT * FROM Customers;
```

The following example uses a multi-line comment to ignore many statements:

Example

```
/*SELECT * FROM Customers;
SELECT * FROM Products;
SELECT * FROM Orders;
SELECT * FROM Categories;*/
SELECT * FROM Suppliers;
```

To ignore just a part of a statement, also use the /* */ comment.

The following example uses a comment to ignore part of a line:

Example

```
SELECT CustomerName, /*City,*/ Country FROM Customers;
```

The following example uses a comment to ignore part of a statement:

Example

```
SELECT * FROM Customers WHERE (CustomerName LIKE 'L%'
OR CustomerName LIKE 'R%' /*OR CustomerName LIKE 'S%'
OR CustomerName LIKE 'T%'*/ OR CustomerName LIKE 'W%')
AND Country='USA'
ORDER BY CustomerName;
```

5.5 SELECT

The SQL SELECT Statement:

The SELECT statement is used to select data from a database.

Example

Return data from the Customers table:

SELECT CustomerName, City FROM Customers;

Syntax

SELECT *column1*, *column2*, ...
FROM *table_name*;

Here, column1, column2, ... are the *field names* of the table you want to select data from.

The table_name represents the name of the *table* you want to select data from.

Select ALL columns:

If you want to return all columns, without specifying every column name, you can use the SELECT * syntax:

Example

Return all the columns from the Customers table:
SELECT * FROM Customers;

5.6 WHERE Clause

The WHERE clause is used to filter records.

It is used to extract only those records that fulfill a specified condition.

Example:

Select all customers from Mexico:

SELECT * FROM Customers
WHERE Country='Mexico';

Syntax

SELECT *column1*, *column2*, ...
FROM *table_name*
WHERE *condition*;

Note: The WHERE clause is not only used in SELECT statements, it is also used in UPDATE, DELETE, etc.!

5.7 DELETE

The SQL DELETE Statement

The DELETE statement is used to delete existing records in a table.

DELETE Syntax

DELETE FROM *table_name* WHERE *condition*;

DELETE Example:

The following SQL statement deletes the customer "Alfreds Futterkiste" from the "Customers" table:

Example:
DELETE FROM Customers WHERE CustomerName='Alfreds Futterkiste';

5.8 UPDATE

The UPDATE statement is used to modify the existing records in a table.

UPDATE Syntax:

UPDATE *table_name*
SET *column1 = value1, column2 = value2, ...*
WHERE *condition*;

UPDATE Table:

The following SQL statement updates the first customer (CustomerID = 1) with a new contact person *and* a new city.

Example:
UPDATE Customers
SET ContactName = 'Alfred Schmidt', City= 'Frankfurt'
WHERE CustomerID = 1;

5.9 Connect Python to MySQL

To test if the installation was successful, or if you already have "MySQL Connector" installed, create a Python page with the following content:

demo_mysql_test.py:

```
import  mysql.connector
```

- If the above code was executed with no errors, "MySQL Connector" is installed and ready to be used.

Create Connection

Start by creating a connection to the database.

Use the username and password from your MySQL database:

demo_mysql_connection.py:

```
import mysql.connector
mydb = mysql.connector.connect(
  host="localhost",
  user="yourusername",
  password="yourpassword"
)

print(mydb)
```

5.10 Flask INTRODUCTION

Flask is a lightweight and flexible web framework for Python. It's designed to make getting started with web development quick and easy, while still being powerful enough to build complex web applications.

What is Flask?

Flask is an API of Python that allows us to build web applications. It was developed by Armin Ronacher. Flask's framework is more explicit than Django's framework and is also easier to learn because it has less base code to implement a simple web application. Flask Python is based on the WSGI(Web Server Gateway Interface) toolkit and Jinja2 template engine.

Advantages of Flask

- Flask is **a lightweight** backend framework with minimal dependencies.
- Flask **is easy to learn** because its simple and intuitive API makes it easy to learn and use for beginners.

Flask is **a flexible Framework** because it allows you to customize and extend the framework to suit your needs easily.

- Flask can be used with **any database** like:- SQL and NoSQL and with **any Frontend Technology** such as React or Angular.
- Flask is **great for small to medium projects** that do not require the complexity of a large framework.
- Flask Documentation.

Getting Started With Flask

Python3 is required for the installation of the Python Web Framework Flask. You can start by importing Flask from the Flask Python package on any Python IDE. For installation on any environment, you can click on the installation link given below. To test that if the installation is working, check out the code given below.

```
from flask import Flask
app = Flask(__name__)   # Flask constructor

# A decorator used to tell the application
# which URL is associated function
@app.route('/')
def hello():
```

```
return 'HELLO'

if __name__=='__main__':
app.run()
```

- The '/' URL is bound with hello() function. When the home page of the webserver is opened in the browser, the output of this function will be rendered accordingly. The Flask Python application is started by calling the run() function. The method should be restarted manually for any change in the code. To overcome this, the debug support is enabled so as to track any error.

```
app.debug = True
app.run()
app.run(debug = True)
```

Build Flask Routes in Python

Nowadays, the web frameworks provide routing technique so that user can remember the URLs. It is useful to access the web page directly without navigating from the Home page. It is done through the following route() decorator, to bind the URL to a function.

```
# decorator to route URL
@app.route('/hello')

# binding to the function of route
def hello_world():
return 'hello world'
```

If a user visits http://localhost:5000/hello URL, the output of the hello_world() function will be rendered in the browser. The add_url_rule() function of an application object can also be used to bind URL with the function as in above example.

```
def hello_world():
return 'hello world'
app.add_url_rule('/', 'hello', hello_world)
```

Variables in Flask:

The Variables in thePython Web Framework flask is used to build a URL dynamically by adding the variable parts to the rule parameter. This variable part is marked as. It is passed as keyword argument. See the example below

```
from flask import Flask
app = Flask(__name__)

# routing the decorator function hello_name
@app.route('/hello/<name>')
def hello_name(name):
return 'Hello %s!' % name

if __name__== '_main_':
app.run(debug = True)
```

Save the above example as hello.py and run from power shell. Next, open the browser and enter the URL http://localhost:5000/hello/GeeksforGeeks.

Output

Hello GeeksforGeeks!

Build a URL in Flask

Dynamic Building of the URL for a specific function is done using url_for() function. The function accepts the name of the function as first argument, and one or more keyword arguments. See this example

```
from flask import Flask, redirect, url_for
app = Flask(_name_)
@app.route('/admin')  # decorator for route(argument) function
def hello_admin():  # binding to hello_admin call
return 'Hello Admin'
@app.route('/guest/<guest>')
def hello_guest(guest):  # binding to hello_guest call
return 'Hello %s as Guest' % guest
@app.route('/user/<name>')
def hello_user(name):
if name == 'admin':  # dynamic binding of URL to function
return redirect(url_for('hello_admin'))
else:
return redirect(url_for('hello_guest', guest=name))
if __name__== '__main__':
app.run(debug=True)
```

To test this, save the above code and run through python shell and then open browser and enter the following URL:-

Inputfile**:** http://localhost:5000/user/admin

Output: Hello Admin

Input: http://localhost:5000/user/ABC

Output: Hello ABC as Guest

The above code has a function named user(name), accepts the value through input URL. It checks that the received argument matches the ‘admin’ argument or not. If it matches, then the function hello_admin() is called else the hello_guest() is called.

HTTP method are provided by Flask

Python Web Framework Flask support various HTTP protocols for data retrieval from the specified URL, these can be defined as:-

Method	Description

Method	Description
GET	This is used to send the data in an without encryption of the form to the server.
HEAD	provides response body to the form
POST	Sends the form data to server. Data received by POST method is not cached by server.
PUT	Replaces current representation of target resource with URL.
DELE TE	Deletes the target resource of a given URL

5.11 Flask App Routing

App Routing means mapping the URLs to a specific function that will handle the logic for that URL.

Example: In our application, the URL ("/") is associated with the root URL. So if our site's domain was www.example.org and we want to add routing to "www.example.org/hello", we would use "/hello".

To bind a function to an URL path we use the *app.route* decorator. In the below example, we have implemented the above routing in the flask

```
from flask import Flask

app = Flask(__name__)

# Pass the required route to the decorator.

@app.route("/hello") def hello():

return "Hello, Welcome toGeeksForGeeks"

@app.route("/")

def index():

return "Homepage of GeeksForGeeks"

if _name____== "_main_":
```

```
app.run(debug=True)
```

The hello function is now mapped with the "/hello" path and we get the output of the function rendered on the browser.

Step to run the application: Run the application using the following command.
python main.py

Output: Open the browser and visit *127.0.0.1:5000/hello*, you will see the following output

5.12 URL Building

- The URLs (Uniform Resource Locators) is fundamental for navigating the internet and accessing various web resources.
- Web Browsing
- Enter the URL into your web browser's address bar and press Enter to visit a webpage. Ensure you include the protocol "**http:// or https://"** for accuracy, though most modern browsers automatically correct this.
- Hyperlinks
- Click on hyperlinks embedded in web pages, emails, or documents to automatically open the linked URL in your web browser.
- Bookmarking
- Save URLs as bookmarks in your browser for easy access to frequently visited sites without needing to remember the URL.
- Sharing
- Share URLs with others via email, messaging apps, or social media to direct them to specific web pages or resources.

Components of a URL

- A website, a URL is basically where the website lives online and it helps visitors to identify the site easily as well as get an idea about its contents.
- A typical website has at least 3 parts in its URL like *www.google.com* but some complex URLs might also have 8 to 9 parts namely scheme, subdomain, domain name, top-level domain, port number, path, query, parameters, and fragment.

Parts of a URL

URL : https://www.example.co.uk:443/blog/article/search?docid=720&hl=en#dayone

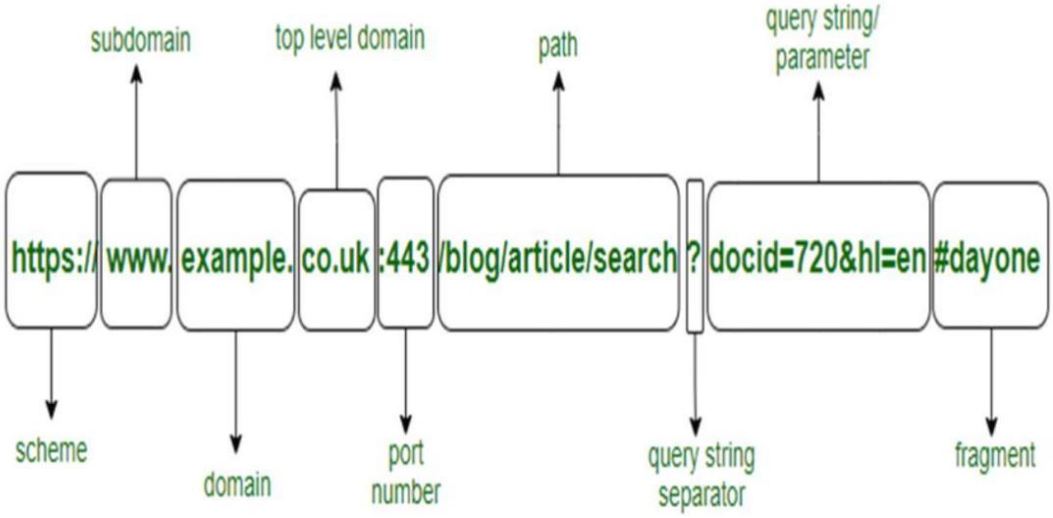


Fig: Components of a URL

1. Scheme :

https://

The protocol or scheme part of the URL and indicates the **set of rules that will decide the transmission and exchange of data**. HTTPS which stands for Hyper Text Transfer Protocol Secure tells the browser to display the page in Hyper Text **(HTML) format** as well as encrypt any information that the user enters in the page. Other protocols include the **FTP** or File Transfer Protocol which is used for transferring files between client and server, **SMTP** or Single Mail Transfer Protocol which is used for sending emails.

2. Subdomain :

https://***www.***

The subdomain is used to separate different sections of the website as it specifies the type of resource to be delivered to the client. Here the subdomain used 'www' is a general symbol for any resource on the web. Subdomains like 'blog' direct to a blog page, 'audio' indicates the resource type as audio.

3. Domain Name :

https://www.***example.***

Domain name specifies the organization or entity that the URL belongs to. Like in *www.facebook.com* the domain name 'facebook' indicates the organization that owns the site.

4. Top-level Domain :

https://www.example.***co.uk***

The TLD (top-level domain) indicates the type of organization the website is registered to. Like the **.com** in *www.facebook.com* indicates a commercial entity. Similarly, .org indicates organization, .co.uk a commercial entity in the UK.

5. Port Number :

https://www.example.co.uk***:443***

A port number specifies the type of service that is requested by the client since servers often deliver multiple services. Some default port numbers include 80 for HTTP and 443 for HTTPS servers.

6. Path :

https://www.example.co.uk:443***/blog/article/search***

Path specifies the exact location of the web page, file, or any resource that the user wants access to. Like here the path indicates a specific article in the blog webpage.

7. Query String :

https://www.example.co.uk:443/blog/article/search?***docid=720&hl=en***

The query string specifies the parameters of the data that is being queried from a website's database. Each query string is made up of a parameter and a value joined by the equals (=) sign. In case of multiple parameters, query strings are joined using the ampersand (&) sign. The parameter can be a number, string, encrypted value, or any other form of data on the database.

8. Fragment :

https://www.example.co.uk:443/blog/article/search?docid=720&hl=en***#dayone***

The fragment identifier of a URL is optional, usually appears at the end, and begins with a hash (#). It indicates a specific location within a page such as the 'id' or 'name' attribute for an HTML element.

5.13 HTTP Methods

What is HTTP?

- The Hypertext Transfer Protocol (HTTP) is designed to enable communications between clients and servers.
- HTTP works as a request-response protocol between a client and server.
- Example: A client (browser) sends an HTTP request to the server; then the server returns a response to the client. The response contains status information about the request and may also contain the requested content.

HTTP Methods

- **GET**
- **POST**
- **PUT**
- **HEAD**
- **DELETE**
- **PATCH**
- **OPTIONS**
- **CONNECT**
- **TRACE**

- **The two most common HTTP methods are: GET and POST.**

The GET Method

- GET is used to request data from a specified resource.
- Note that the query string (name/value pairs) is sent in the URL of a GET request: /test/demo_form.php?name1=value1&name2=value2

Some notes on GET requests:

- GET requests should never be used when dealing with sensitive data
- GET requests have length restrictions
- GET requests are only used to request data (not modify)

The POST Method

- POST is used to send data to a server to create/update a resource.
- The data sent to the server with POST is stored in the request body of the HTTP request:
- POST /test/demo_form.php HTTP/1.1
  name1=value1&name2=value2

Some notes on POST requests:

- POST requests are never cached
- POST requests do not remain in the browser history
- POST requests cannot be bookmarked
- POST requests have no restrictions on data length

The PUT Method:

- PUT is used to send data to a server to create/update a resource.

- The difference between POST and PUT is that PUT requests are idempotent. That is, calling the same PUT request multiple times will always produce the same result. In

contrast, calling a POST request repeatedly have side effects of creating the same resource multiple times.

The HEAD Method

- HEAD is almost identical to GET, but without the response body.
- In other words, if GET /users returns a list of users, then HEAD /users will make the same request but will not return the list of users.
- A HEAD request is useful for checking what a GET request will return before actually making a GET request - a HEAD request can read the Content-Length header to check the size of the file, without actually downloading the file.

The DELETE Method

- The DELETE method deletes the specified resource.

The PATCH Method

- The PATCH method is used to apply partial modifications to a resource.

The OPTIONS Method

- The OPTIONS method describes the communication options for the target resource.

The CONNECT Method

- The CONNECT method is used to start two-way communications (a tunnel) with the requested resource.

The TRACE Method

- The TRACE method is used to perform a message loop-back test that tests the path for the target resource (useful for debugging purposes).

5.14 Templates

In Python, **templates** are used to separate the presentation layer (what users see on a webpage) from the logic layer (how data is processed). Templates allow us to create dynamic content by injecting data from Python code into HTML files.

There are two ways:

- Python code handles **data** and **logic** (like fetching data from a database or performing calculations).
- Templates handle **presentation** (how that data is displayed to the user in a webpage).

For example, instead of writing Python code directly inside HTML files, we can use templates to embed Python variables and control structures (like loops and conditions) in HTML, making the code cleaner and easier to manage.

Why Do We Need Templates?

1. **Separation of Concerns**:
   - Templates help you separate the code that handles **logic** (Python) from the **presentation** (HTML).
   - This makes your code easier to read, maintain, and debug.
2. **Reusability**:
   - With templates, you can reuse common structures (like headers, footers, or sidebars) across multiple pages.
3. **Dynamic Content**:
   - Templates allow you to inject dynamic data (such as user information, database records, etc.) into your web pages.

How Templates Work in Python (with Flask)

Flask is a popular Python web framework that uses the **Jinja2** template engine. Jinja2 allows us to use placeholders in HTML, which are then replaced with real data when the page is rendered.

Step-by-Step Example

1. **Setting Up the Project**

Create a simple project structure like this:

pgsql
Copy
```
my_project/
├── app.py
└── templates/
└── index.html
```

2. **Python Code (app.py)**

In the Python file, we create a Flask application. This application will send data to the template for rendering.

python

Copy
```
from flask import Flask, render_template

app = Flask(_name_)

@app.route('/')
def index():
title = "Welcome to My Website"
users = ["Alice", "Bob", "Charlie"]
return render_template('index.html', title=title, users=users)

if __name__== "__main__":
app.run(debug=True)
```

In this code:

- render_template('index.html', title=title, users=users):
  - This tells Flask to look for an index.html template and send the title and users data to it.

3. **HTML Template (index.html)**

In the templates/ folder, we create an index.html template file:

html
Copy
```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>{{ title }}</title>
</head>
<body>
<h1>{{ title }}</h1>
<ul>
{% for user in users %}
<li>{{ user }}</li>
{% endfor %}
</ul>
</body>
</html>
```

In the template:

- **{{ title }}**: Jinja2 will replace this with the title variable from Python.

- **{% for user in users %} ... {% endfor %}**: This loop iterates over the users list and displays each user's name in a <li> tag.

4. **Run the Application**

Now, run the Python file (app.py) using:

bash
Copy
```
python app.py
```

Go to http://127.0.0.1:5000/ in your browser, and you'll see the webpage rendered with dynamic content (the list of users).

Key Features of Jinja2 Templates

1. **Variables**
   - You can insert Python variables into HTML using **double curly braces** {{ }}.

     html
     Copy
     ```
     <h1>{{ username }}</h1>
     ```

2. **Loops**
   - You can loop through data (e.g., lists or dictionaries) with {% for %}.

     html
     Copy
     ```
     {% for item in items %}
     <p>{{ item }}</p>
     {% endfor %}
     ```

3. **Conditions**
   - You can use if statements to display content based on conditions.

     html
     Copy
     ```
     {% if user.is_logged_in %}
     <p>Welcome back, {{ user.name }}!</p>
     {% else %}
     <p>Please log in.</p>
     {% endif %}
     ```

4. **Filters**
   - Filters modify the output, such as formatting a date or changing text case.

html
Copy
```
{{ name|capitalize }}  <!-- Capitalizes the name -->
{{ price|round(2) }}   <!-- Rounds a price to 2 decimal places -->
```

5. **Template Inheritance**
   - You can create a base template with common parts (like header, footer) and extend it in other templates.
   - **Base template (base.html)**:

     html
     Copy
     ```
     <!DOCTYPE html>
     <html>
     <head>
     <title>{% block title %}My Website{% endblock %}</title>
     </head>
     <body>
     <header>
     <h1>Header Content</h1>
     </header>

     <div id="content">
     {% block content %}{% endblock %}
     </div>

     <footer>
     <p>Footer Content</p>
     </footer>
     </body>
     </html>
     ```

   - **Child template (index.html)**:

     html
     Copy
     ```
     {% extends "base.html" %}

     {% block title %}Home - My Website{% endblock %}

     {% block content %}
     <h2>Welcome to the Home Page!</h2>
     {% endblock %}
     ```

5.15 Request Body

- The requests module allows you to send HTTP requests using Python.

- The HTTP request returns a Response Object with all the response data (content, encoding, status, etc).

Syntax

requests.*methodname(params)*

Example:

```
import requests

x = requests.get('https://w3schools.com/python/demopage.html')

print(x.text)
```

Methods:

Method	Description
delete(*url, args*)	Sends a DELETE request to the specified url
get(*url, params, args*)	Sends a GET request to the specified url
head(*url, args*)	Sends a HEAD request to the specified url
patch(*url, data, args*)	Sends a PATCH request to the specified url
post(*url, data, json, args*)	Sends a POST request to the specified url

put(*url*, *data*, *args*)	Sends a PUT request to the specified url
request(*method*, *url*, *args*)	Sends a request of the specified method to the specified url

5.16 Developing Web Applications connecting My SQL.

- Developing web applications that connect to a MySQL database in Python typically involves a combination of a web framework (like Flask or Django), a MySQL connector library (like mysql-connector-python or PyMySQL), and a database setup. Here's a step-by-step breakdown of how you can go about creating such applications**: MySQL Database.**

Step 1: Setting Up MySQL

Before developing the web application, you need to set up a MySQL database.

1. **Install MySQL**: You can download MySQL from here.
2. **Create a Database**: After installing MySQL, you can create a database using the MySQL command line

   sql
   Copy
   CREATE DATABASE my_database;

3. **Create Tables**: You'll also need to define the tables you want to use in your web application.

   Example:

   sql
   Copy
   CREATE TABLE users (
   id INT AUTO_INCREMENT PRIMARY KEY,
   username VARCHAR(255) NOT NULL,
   email VARCHAR(255) NOT NULL,
   password VARCHAR(255) NOT NULL
   );

Step 2: Install Necessary Python Libraries

To interact with a MySQL database in Python, we need to install a MySQL connector. we can install the following libraries using pip:

1. **Install Flask**: If you are using Flask as the web framework.

   bash
   Copy
   pip install Flask

2. **Install MySQL Connector**: There are a couple of MySQL connectors in Python:
   - **mysql-connector-python**:

     bash
     Copy
     pip install mysql-connector-python

   - **PyMySQL**:

     bash
     Copy
     pip install PyMySQL

3. Both libraries allow Python to interact with MySQL databases.

Step 3: Set Up the Web Framework (Flask Example)

Let's use **Flask**, a lightweight Python web framework, for this example. Flask makes it easy to develop simple web applications quickly.

1. **Create a basic Flask app**:

   First, create a new directory for your project and a Python file (e.g., app.py).

   python
   Copy
   ```
   from flask import Flask, render_template, request, redirect, url_for
   import mysql.connector

   app = Flask(__name__)

   # Connect to MySQL
   def get_db_connection():
   connection = mysql.connector.connect(
   host='localhost',
   ```

```
user='your_mysql_user',
password='your_mysql_password',
database='my_database'
)
return connection

@app.route('/')
def index():
conn = get_db_connection()
cursor = conn.cursor()
cursor.execute('SELECT * FROM users')
users = cursor.fetchall()
conn.close()
return render_template('index.html', users=users)

@app.route('/add', methods=['POST'])
def add_user():
username = request.form['username']
email = request.form['email']
password = request.form['password']

conn = get_db_connection()
cursor = conn.cursor()
cursor.execute('INSERT INTO users (username, email, password) VALUES (%s, %s,
%s)', (username, email, password))
conn.commit()
conn.close()

return redirect(url_for('index'))

if _name___== '_main_':
app.run(debug=True)
```

In this example:

- **get_db_connection**: This function handles the connection to MySQL.
- **index**: The root endpoint (/) fetches data from the users table and displays it in an HTML template.
- **add_user**: This endpoint handles POST requests to add a new user to the database.

2. **Create an HTML Template**:

   Flask uses the Jinja2 templating engine. You will need to create an index.html file inside a folder called templates.

   html

Copy

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>MySQL Flask Example</title>
</head>
<body>
<h1>User List</h1>
<table border="1">
<thead>
<tr>
<th>Username</th>
<th>Email</th>
</tr>
</thead>
<tbody>
{% for user in users %}
<tr>
<td>{{ user[1] }}</td>
<td>{{ user[2] }}</td>
</tr>
{% endfor %}
</tbody>
</table>

<h2>Add User</h2>
<form action="/add" method="POST">
<label for="username">Username:</label>
<input type="text" name="username" required>
<br>
<label for="email">Email:</label>
<input type="email" name="email" required>
<br>
<label for="password">Password:</label>
<input type="password" name="password" required>
<br>
<button type="submit">Add User</button>
</form>
</body>
</html>
```

This HTML template renders the users' information in a table and provides a form for adding new users to the database.

Step 4: Run the Application

Now you can run your Flask application by running the following command in your terminal:

bash
Copy
```
python app.py
```

By default, Flask runs on http://127.0.0.1:5000/. You can visit this URL in your browser, and you should see the list of users from the MySQL database and a form to add new users.

Step 5: Error Handling and Optimizations

When building a production application, consider the following:

1. **Error Handling**: Always handle possible exceptions like connection errors, SQL execution errors, etc. You can use try-except blocks to catch exceptions and return friendly error messages.
2. **SQL Injection Prevention**: Use parameterized queries (as shown in the example) to prevent SQL injection attacks. Never directly interpolate user inputs into SQL queries.
3. **Environment Variables**: Store your MySQL credentials (like username, password, and database name) in environment variables instead of hardcoding them in the script for security reasons.
4. **ORM (Object-Relational Mapping)**: For complex applications, you might want to use an ORM (which works with both Flask and Django). ORMs abstract away raw SQL, making it easier to work with databases and more scalable for larger applications.